



U2U

TRAINING

On-schedule in Brussels. On-site in Europe & Middle East.

Developing web apps with React



Developing web apps with React

Version: January 2019

**Visit the U2U website
www.u2u.be**

Copyright © 2000-2019 U2U Training nv/sa, B-1731 Zellik (Brussels), Belgium.

Niets uit dit document mag worden verveelvoudigd en/of openbaar gemaakt door middel van druk, fotokopie, microfilm, elektronisch of op welke andere wijze ook en evenmin in een retrieval systeem worden opgeslagen zonder voorafgaande schriftelijke toestemming van de auteur.

Hoewel dit document met zeer veel zorg is samengesteld, aanvaarden de auteurs geen enkele aansprakelijkheid voor schade ontstaan door eventuele fouten en/of onvolkomenheden in dit document.

Copyright © 2000-2019 U2U Training nv/sa, B-1731 Zellik (Brussels), Belgium.

No part of this document may be reproduced or transmitted in any form or by any means, electronic or mechanical, for any purpose, without the express written permission of the author.

This publication is provided "as is" without warranty of any kind, either express or implied, including, but not limited to, the implied warranties of merchantability, fitness for a particular purpose, or non-infringement.

This publication could include technical inaccuracies or typographical errors. Changes are periodically added to the information herein; these changes will be incorporated in new editions of the publication.

Published by U2U Training nv/sa, B-1731 Zellik (Brussels), Belgium

1 TABLE OF CONTENTS

1	TABLE OF CONTENTS	3
2	WHAT IS REACT JS.....	7
2.1	Modern Web Applications	8
2.2	What is React?	10
2.3	Components.....	11
3	REACT FUNDAMENTALS.....	13
3.1	React App Structure	14
3.2	What is JSX?	17
3.3	JavaScript Expressions	20
3.4	JSX Tips and Tricks.....	24
3.5	Summary	27
3.6	Lab – Hello World	28
3.6.1	Before you start.....	28
3.6.2	Creating the project	28
3.6.3	Running the application	29
3.6.4	Updating the template	29
3.6.5	Adding a component.....	30
3.6.6	Creating a static template for the component	30
3.6.7	Adding in real data	32
3.6.8	(Optional) Extra challenge.....	34
4	COMPONENTS	35
4.1	State Machines	36
4.2	Component State	37
4.3	Interactivity	40

4.4	Multiple Components.....	44
4.5	Transferring props.....	48
4.6	Form Components.....	50
4.7	Alternative Syntax.....	52
4.8	Summary	53
4.9	Lab – Todo List	54
4.9.1	Before you start.....	54
4.9.2	Creating the project	54
4.9.3	Getting some overhead out of the way	54
4.9.4	Creating the components.....	55
4.9.5	Adding in the state	57
4.9.6	Making the application interactive	60
4.9.7	(Optional) extra challenge: Make the delete button work	64
5	COMPONENT LIFECYCLE AND PERFORMANCE.....	65
5.1	Component Lifecycle	66
5.2	Advanced Performance	68
5.3	PureComponent.....	75
6	STRUCTURING APPLICATIONS TOWARDS COMPONENTS	76
6.1	Thinking in React.....	77
6.2	Lab – Creating a webshop With React	85
6.2.1	Before you start.....	85
6.2.2	Check out your mock.....	85
6.2.3	Splitting up the mock into components.....	86
6.2.4	Building a static version	87
6.2.4.1	Creating the classes	87
6.2.4.2	Adding in the Templates	88
6.2.5	Adding in state	91

6.2.5.1	Defining the shape of the state.....	91
6.2.5.2	Where does the state live?	92
6.2.6	Creating the inverse data flow A.K.A. Adding interactivity	94
6.2.7	(Optional) Extra Challenge: Don't show an empty Cart	98
7	SPA WITH REACT	99
7.1	Towards Single Page Applications	100
7.2	Introducing React Router	101
7.3	Nested Routes.....	107
7.4	Advanced Usage.....	109
7.5	Lab – Building a SPA with React and React-Router	112
7.5.1	Before you start.....	112
7.5.2	Installing react-router	112
7.5.3	Adding a Navbar	113
7.5.4	Enabling the router	113
7.5.5	Navigating the courses.....	114
7.5.6	Showing Courses by Category.....	117
7.5.7	(Optional) Extra challenge: allowing back-navigation.....	118
8	REACT INTERNALS	119
8.1	Refs	120
8.2	Ref Callback	121
8.3	Property Types	124
8.4	Default Prop Values.....	128
9	REDUX	129
9.1	What is Redux?	130
9.2	Actors	132
9.3	Tools.....	137

9.4	Summary	138
9.5	Lab – Creating a webshop With React And Redux	139
9.5.1	Before you start.....	139
9.5.2	Installing Redux	139
9.5.3	Creating the reducers.....	139
9.5.3.1	Defining the action types	139
9.5.3.2	Creating actions for the action types.....	141
9.5.4	Creating the reducers.....	141
9.5.5	Creating the store	143
9.5.6	Hooking up the store.....	144
9.5.7	tying everything together	145
10	JEST	148
10.1	What is Jest	149
10.2	Creating Tests	150
10.3	Asynchronous Methods.....	152
10.4	Mocking	155
10.5	Jest and React.....	156

2 WHAT IS REACT JS

Agenda



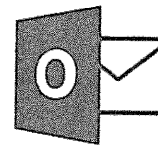
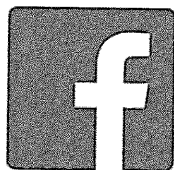
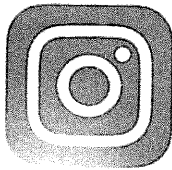
- Introducing Modern Web Applications
- What is React?
- Other libraries
- Components

2.1 MODERN WEB APPLICATIONS

Modern Web Applications

u2u

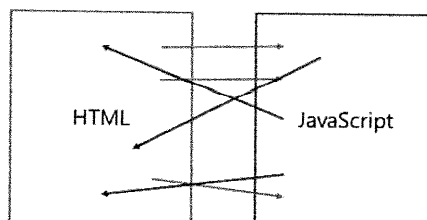
- Heavy focus on the client
- Rich user experience
- Server is mainly for data
- Requires front-end JavaScript framework



Web Applications

u2u

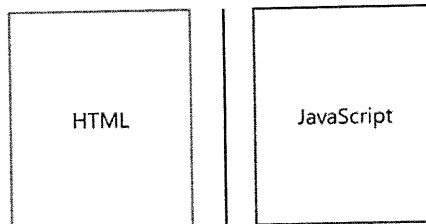
- First Version
 - Heavy intertwined code
 - HTML and JavaScript impact each other
 - change in HTML leads to change in JS
 - Bad for maintenance, testing, ...
 - Native JavaScript and jQuery



HTML5 Brought a New Era...

u2u

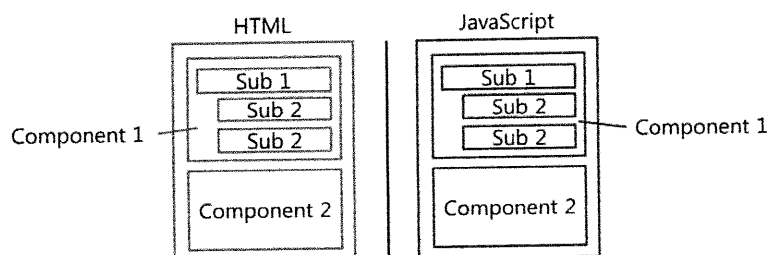
- Separated Code thanks to Data Binding
→ both can evolve without heavy impact on the other
- Better for maintenance, testing, ...
- Allows creation of SPAs (Single Page Applications)
- Backbone, Knockout, Angular 1, Ember



Welcome to the Future

u2u

- Component-based Development
- Only using Data Binding can lead to very large and complicated code
- Focus on latest standards for HTML and JS
- ReactJS, Angular



2.2 WHAT IS REACT?

What is React?



- Open Source UI Framework
- React JS is the View and ViewModel in MVVM
 - So what about the rest? Use external libraries!
- For creating Single Page Application using UI Components
- Built by Facebook
- Installation with npm: `npm install -g react`

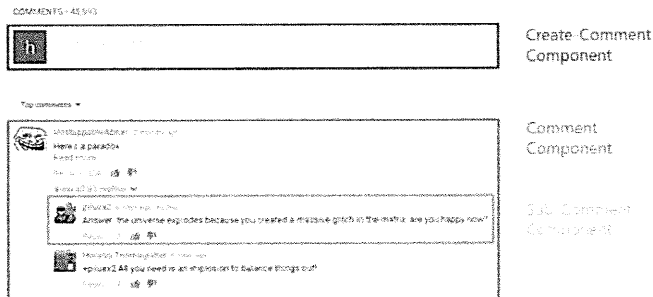


2.3 COMPONENTS

Components



- A **component** is a combination of
 - an HTML **representation**
 - a **class** that manages that representation



Components



- React Doesn't Use Separate Templates
- The React Team created JSX
 - optional but recommended syntax
- JSX needs translation towards JS

```
import React from 'react';

export class ReadMe extends React.Component{
  render(){
    return <h1>Hello World!</h1>;
  }
}
```

Tools



- create-react-app is an npm cli tool to create new React Projects
 - Build process is configured
 - Best practices are used in the template
- React Developer Tools
 - Plugin for Chrome which can help you keep track of your application at runtime
- You can build React apps in your IDE of choice
 - Look for snippets and syntax extensions

3 REACT FUNDAMENTALS

Agenda



- React App Structure
- What is JSX?
 - Html Tags vs React Components
 - Creating Components
- JavaScript Expressions
- JSX Tips and Tricks

3.1 REACT APP STRUCTURE

Creating React App



- The easiest way to create a React app is by using the cli tool
- First, install it through npm: **npm i -g create-react-app**
- Next, open a terminal to the folder where you want to create your app
 - E.g. C:\Users\Bram\source
- Finally, run **create-react-app [project-name]**
 - This might take a couple of minutes...
- When this is finished, you should have an application ready to go

React App Structure



- What are the main aspects of a React app?

```
node_modules
public
  favicon.ico
  index.html
  manifest.json
src
  components
    App.css
    App.js
    App.test.js
  index.css
  index.js
  logo.svg
  registerServiceWorker.js
.gitignore
package-lock.json
package.json
README.md
```

- **node_modules:** dependencies, managed by npm.
- **Public folder:** contains public files like index.html, images and fonts.
- **Components:** the building blocks of your application.
- **index.js:** the entry point into your application.

React App Structure



- What actually happens when you run the application?
 - **index.html** contains an element with a certain id.
 - **index.js** looks for this element and places a component in it.
 - **App.js** is often the main component, it will contain a hierarchy of child components.

React App Structure



```
HTML
<!--index.html-->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-
fit=no">
  <title>React App</title>
</head>

<body>
  <noscript>
    You need to enable JavaScript to run this app.
  </noscript>
  <div id="root"></div>
</body>
</html>
```

This element will be targeted in
index.js

React App Structure

U2U

JS

//index.js

import React from 'react';

Import the necessary React classes

import ReactDOM from 'react-dom';

Import the App component

import App from './App';

ReactDOM.render(<App />, document.getElementById('root'));

Look for the element with id "root" and render the App component inside it

React App Structure

U2U

JS

//App.js

import React, { Component } from 'react';

Import React and the Component class

import logo from './logo.svg';

Import some resources

import './App.css';

Extend the component class

class App extends Component {

render() {

return (

<div className="App">

<header className="App-header">

<h1 className="App-title">Welcome to React</h1>

</header>

</div>

);

}

}

Override the render() method with your template

export default App;

3.2 WHAT IS JSX?

What is JSX?

U2U

- Let's take a closer look at our component's render method.
 - The method returns html, but we aren't putting it in a string.
 - We're using interpolation to set the src attribute for our logo.

App.jsx

```
class App extends Component {  
  render() {  
    return (  
      <div className="App">  
        <header className="App-header">  
          <img src={logo} className="App-logo" alt="logo" />  
          <h1 className="App-title">Welcome to React</h1>  
        </header>  
      </div>  
    );  
  }  
}
```

- We can do this because of JSX!

What is JSX?

U2U

- JavaScript syntax extension
- Developed by React team
- Concise and familiar syntax
- Transpiles into JavaScript
- Creates a visual divide between logic and markup for the developer

Parsers and Transpilers



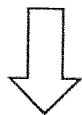
- JSX cannot be interpreted by browsers directly
 - So we need a parser or transpiler to help us
- Parser:
 - Allows JSX to be executed and interpreted at runtime
 - E.g.: babel-runtime
- Transpiler:
 - Translates JSX to JS during compilation
 - E.g.: babel

JSX example



- After translation JSX turns into JavaScript
 - Uses `React.createElement`

```
// Input (JSX):  
var app = <Nav color="blue" />;
```



```
// Output (JS):  
var app = React.createElement(Nav, {color:"blue"});
```

HTML Tags vs. React Components



- React allows HTML tags and React components
- Use lower-case to render an HTML tag:

```
var myDivElement = <div className="foo" />;
ReactDOM.render(myDivElement, document.getElementById('ex'));
```

- Use PascalCasing to render a React Component:

```
class MyComponent extends React.Component{/*...*/}
var myElement = <MyComponent someProperty={true} />;
ReactDOM.render(myElement, document.getElementById('example'));
```

- React's JSX uses this convention to distinguish between local component classes and HTML tags.

Creating Components



- JSX is not mandatory.
 - You can create components using plain JS as well.
 - But it becomes harder to reason about.

```
class Hello extends React.Component {
  render() {
    return React.createElement('div', null, `Hello ${this.props.toWhat}`);
  }
}

ReactDOM.render(
  React.createElement(Hello, {toWhat: 'World'}, null),
  document.getElementById('root')
);
```

3.3 JAVASCRIPT EXPRESSIONS

JavaScript Expressions



- The `render()` method of a component returns HTML
 - Uses the JSX notation
- Static HTML is fine, but we need dynamic behavior!
- Luckily JSX allows JavaScript
 - Uses the (single) mustache notation
 - Contains JavaScript Expressions



Simple Expressions



- The `render()` method is just that, a method
 - We can add variables and use them in our templates

```
export class PizzaDetail extends Component{
  render(){
    let pizza = { name: "Margherita", price: 8, spicy: false};

    return <ul>
      <li>{pizza.name} - {pizza.price} - Is it spicy? {pizza.spicy}</li>
    </ul>;
  }
}
```

Property Binding



- Assign an expression to a property
 - Mustaches are also used here

```
export class PizzaDetail extends Component{
  render(){
    let pizza = { id: 1, name: "Margherita", price: 8,
                  spicy: false};
    return <ul>
      <li data-id={pizza.id}>
        {pizza.name} - {pizza.price} - spicy?{pizza.spicy}
      </li>
    </ul>
  }
}
```

Function Calls in JSX



- The JavaScript expressions in JSX should always return a primitive value
 - Use functions, ternary operators (?:)
 - Don't return an object (not allowed) – unless it's a component

```
export class PizzaDetail extends Component{

  pizzaToString(pizza){
    return `${pizza.name} - ${pizza.price} - Is it spicy? ${pizza.spicy}`;
  }

  render(){
    let pizza = { id: 1, name: "Margherita", price: 8, spicy: false};
    return <ul>
      <li data-id={pizza.id}>{this.pizzaToString(pizza)}</li>
    </ul>
  }
}
```

Boolean Values



- Using an attribute with no value causes JSX to put 'true'
- Used often with HTML form elements
 - Disabled, required, checked, readonly, ...

```
// These two are equivalent in JSX for disabling a button
<input type="button" disabled />;
<input type="button" disabled={true} />;

// And these two are equivalent in JSX for enabling a button
<input type="button" />;
<input type="button" disabled={false} />;
```

Showing Collections



- Use the Array.map() function

```
export class PizzaDetail extends Component{
  pizzaToString(pizza){
    return `${pizza.name} - ${pizza.price} - Is it spicy? ${pizza.spicy}`;
  }

  render(){
    let pizzas = [{ id: 1, name: "Margherita", price: 8, spicy: false},
                  { id: 2, name: "Napoletana", price: 10, spicy: true }];
    let listItems = pizzas.map((pizza) => <li data-id={pizza.id}>
                                {this.pizzaToString(pizza)}
                                </li>);

    return <ul>
      {listItems}
    </ul>
  }
}
```



Child Components

- Create child components in an expression

```
class App extends Component {
  render() {
    return (
      <div className="App">
        <div className="App-header">
          {window.isLoggedIn ? <GameCreate /> : <Login />}
        </div>
        <div className="App-intro">
          <GameTable />
        </div>
      </div>
    );
  }
}
```



Comments

- How do you add comments in JSX?
 - Put {} around the comments inside **content**

```
return (
  <Nav>
    { /* inside content, put {} around */ }
    <Person
      /* multi
      line
      comment */
      name={window.isLoggedIn ? window.name : ''} // end of line
    />
  </Nav>
);
```

3.4 JSX TIPS AND TRICKS

JSX Tips and Tricks

U2U

- JSX is not entirely like plain HTML
- There are some catches:
 - JSX / React-specific rules
 - Default HTML attributes
 - HTML Entities
 - Custom HTML Attributes

JSX / React-specific Rules

U2U

- Tags need closing slashes

`<p>Lorem ipsum dolor sit<p><!-- Correct in HTML5, not allowed in JSX -->`

- `render()` returns 1 top-level element
 - Sometimes you want to return multiple top-level elements
 - Use `React.Fragment` (or `<> ... </>`)

```
class Columns extends React.Component {
  render() {
    return (
      <React.Fragment>
        <td>Hello</td>
        <td>World</td>
      </React.Fragment>
    );
  }
}
```

Default HTML attributes



- There are a couple of attributes that need to be changed in order to work
 - class → className
 - colspan → colSpan
 - innerHTML → dangerouslySetInnerHTML
 - for → htmlFor
 - style → style (but with an object!)
 - ...

HTML Entities



- No problems within literal text

```
<div>First &middot; Second</div>
```

- HTML entity within dynamic content
 - Double escaping happens for XSS prevention

```
// Bad: It displays "First &middot; Second"  
<div>{'First &middot; Second'}</div>
```

- Solutions:
 - UTF8 character in JS
 - Use Unicode number
 - mixed arrays with strings and JSX elements
 - Insert raw HTML

HTML Entities



JSX

```
// UTF-8 character: don't forget to set encoding correctly  
<div>{'First · Second'}</div>;
```

```
// use Unicode number
```

```
<div>{'First \u00b7 Second'}</div>;
```

```
<div>{'First ' + String.fromCharCode(183) + ' Second'}</div>;
```

```
// Mixed arrays of strings and jsx elements
```

```
<div>[['First ', <span>&middot;</span>, ' Second']]</div>
```

```
// Raw HTML
```

```
<div dangerouslySetInnerHTML =  
  {__html: 'First &middot; Second'} />
```

3.5 SUMMARY

Summary



- React App Structure
- What is JSX?
 - Html Tags vs React Components
 - Creating Components
- JavaScript Expressions
- JSX Tips and Tricks

3.6 LAB – HELLO WORLD

In this lab you will build a simple React application to show some personal info. The objective of this lab is to get a feel for React apps and learn to use the create-react-app cli-tool

3.6.1 BEFORE YOU START

This lab starts from scratch. You will however need to install some tools and libraries:

NodeJs >= 6 (can be found at <https://nodejs.org/en/>)

Create-react-app: once you have Node installed, you can install this tool through npm:

In a command-line terminal, run the following command

COMMAND LINE

```
npm i -g create-react-app
```

This installs the create-react-app cli tool globally.

3.6.2 CREATING THE PROJECT

Open a command line terminal in your source folder, and run the following command in there:

COMMAND LINE

```
npx create-react-app hello-world
```

When this is done running, you can open the newly created project in the IDE of your choice, you should get a folder structure that looks more or less as follows:



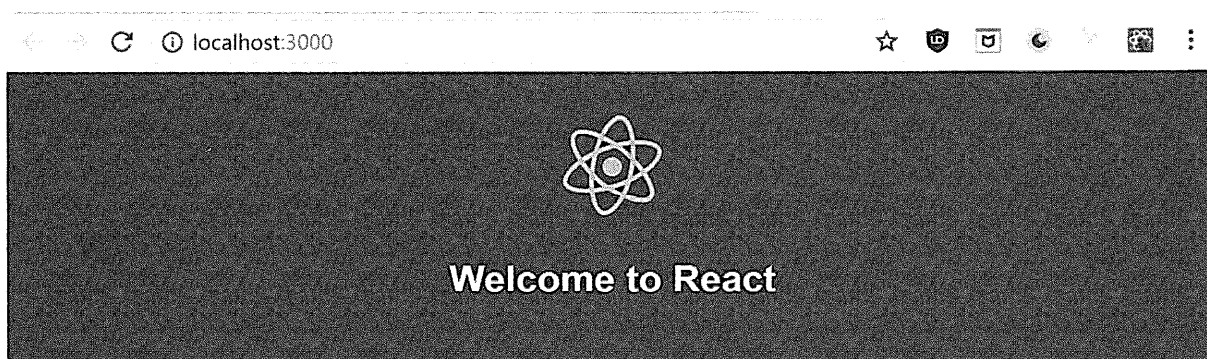
3.6.3 RUNNING THE APPLICATION

Before you change anything, run the application by executing the following command:

COMMAND LINE

```
npm start
```

You should get something like this:



To get started, edit `src/App.js` and save to reload.

3.6.4 UPDATING THE TEMPLATE

When you take a look at `src/App.js`, you should find a **class** called **App** that **extends the Component class** from **React**.

1. **Modify the render method** of your App class so that it returns the following html:

HTML

```
<div className="App">
  <img src={logo} className="App-logo" alt="logo" />
  <h1 className="App-title">Hello, world!</h1>
  <h2>welcome To [name]'s first React App!</h2>
</div>
```

(of course you can replace [name] with your actual name).

3.6.5 ADDING A COMPONENT

You're going to show a card with your personal details inside your application. You might want to create a component for that.

2. Create a new file under **src/UserDetails.jsx**.
3. Import React and Component.
4. Create a class that extends the Component class from React, and add a render method.
5. Export the class as default.

USERDETAILS.JSX

```
import React, { Component } from 'react';

class UserDetails extends Component {
  render() {
    return ;
  }
}

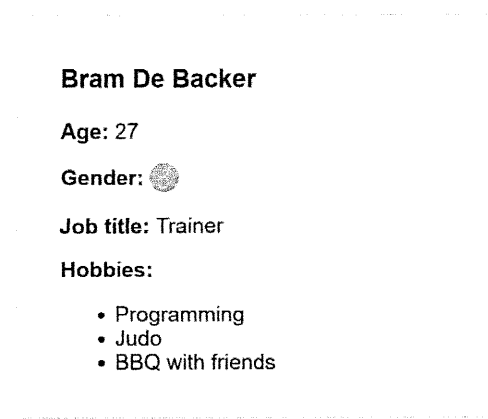
export default UserDetails;
```

3.6.6 CREATING A STATIC TEMPLATE FOR THE COMPONENT

You want to show the following information about yourself inside the component:

- Name
- Age
- Gender
- Job title
- Hobbies

The idea you had in mind was something like this:



6. Update the render method of your app to reflect this.

USERDETAILS.JSX

```
render() {
  return (
    <div className="user-details">
      <h3>Bram De Backer</h3>
      <p><strong>Age: </strong>27</p>
      <p><strong>Gender: </strong><span
        className="gender male">&#9794;</span></p>
      <p><strong>Job title: </strong>Trainer</p>
      <p><strong>Hobbies: </strong></p>
      <ul>
        <li>Programming</li>
        <li>Judo</li>
        <li>BBQ with friends</li>
      </ul>
    </div>);
}
```

7. Of course you don't have to figure out the css for this on your own, just add the following rules to App.css

APP.CSS

```
.user-details{
  width: 300px;
  margin: 0 auto;
  padding: 1em 2em;
  border: 1px solid #CCC;
  border-radius: 5px;
  text-align: left;
}
span.gender{
  color: white;
  padding: 2px 5px;
  border-radius: 12px;
}
span.male{
  background-color: #49C0FF;
}
span.female{
  background-color: #F578FF;
}
```

8. If you're a woman, the utf-8 character code for the female symbol is **♀**

Let's add your new component to the application.

9. In App.js, **render** your **new component**.

APP.JS

```
import React, { Component } from 'react';
import logo from './logo.svg';
import './App.css';
import UserDetails from './UserDetails';

class App extends Component {
  render() {
    return (
      <div className="App">
        <img src={logo} className="App-logo" alt="logo" />
        <h1 className="App-title">Hello, World!</h1>
        <h2>welcome To Bram's first React App!</h2>
        <UserDetails/>
      </div>
    );
  }
}

export default App;
```

Notice the import statement. Without this, react would have no idea what <UserDetails/> means.

You should see your component in the browser.

3.6.7 ADDING IN REAL DATA

Now you probably want to plug your own data into the app. Normally you'd store this in the state of one of your components, but since we didn't talk about that yet, you can just create a simple object inside your render function.

10. Create an object to store your personal data (bear in mind that months are zero-based in JavaScript)

USERDETAILS.JSX

```
render() {
  let me = {
    name: 'Bram De Backer',
    birthDate: new Date(1991, 2, 14),
    gender: 'male',
    jobTitle: 'Trainer',
    hobbies: ['Programming', 'Judo', 'BBQ with friends']
  };
  return (...);
}
```

11. You can already use the mustache syntax to plug in your name and jobtitle, the other fields require some more thinking...

USERDETAILS.JSX

```

render() {
  let me = {
    name: 'Bram De Backer',
    birthDate: new Date(1991, 2, 14),
    gender: 'male',
    jobTitle: 'Trainer',
    hobbies: ['Programming', 'Judo', 'BBQ with friends']
  };

  return (
    <div className="user-details">
      <h3>{me.name}</h3>
      <p><strong>Age: </strong>27</p>
      <p><strong>Gender: </strong><span className="gender
male">#9794;</span></p>
      <p><strong>Job title: </strong>{me.jobTitle}</p>
      <p><strong>Hobbies: </strong></p>
      <ul>
        <li>Programming</li>
        <li>Judo</li>
        <li>BBQ with friends</li>
      </ul>
    </div>);
}

```

12. Let's work top to bottom, starting with your age. Create a helper method to calculate your age in years and call it from the template. (use an arrow function for this to prevent binding problems)

USERDETAILS.JSX

```

class UserDetails extends Component {
  getAge = (birthDate) =>{
    let ageDifMs = Date.now() - birthDate.getTime();
    let ageDate = new Date(ageDifMs); // milliseconds from epoch
    return Math.abs(ageDate.getUTCFullYear() - 1970);
  }

  render() {
    let me = {...};

    return (
      <div className="user-details">
        <h3>{me.name}</h3>
        <p><strong>Age: </strong>{this.getAge(me.birthDate)}</p>
        <p><strong>Gender: </strong><span className="gender
male">#9794;</span></p>
        <p><strong>Job title: </strong>{me.jobTitle}</p>
        <p><strong>Hobbies: </strong></p>
        <ul>
          <li>Programming</li>
          <li>Judo</li>
          <li>BBQ with friends</li>
        </ul>
      </div>);
    }
  }
}

```

13. Next up is your gender. You'll have to set the symbol and class dynamically. Prevent double escaping of the symbol by using one of the technique we saw in this chapter.

USERDETAILS.JSX

```

class UserDetails extends Component {
  ...

  render() {
    let me = {...};

    return (
      <div className="user-details">
        <h3>{me.name}</h3>
        <p><strong>Age: </strong>{this.getAge(me.birthDate)}</p>
        <p><strong>Gender: </strong>
          <span className={"gender "+me.gender}>
            {me.gender === 'male' ?
              String.fromCharCode(9794) :
              String.fromCharCode(9792)}
            </span></p>

        <p><strong>Job title: </strong>{me.jobTitle}</p>
        <p><strong>Hobbies: </strong></p>
        <ul>
          <li>Programming</li>
          <li>Judo</li>
          <li>BBQ with friends</li>
        </ul>
      </div>);
  }
}

```

14. The last hurdle is showing your hobbies. Remember the `Array.map` function? Time to use it to its full potential.

USERDETAILS.JSX

```

class UserDetails extends Component {
  ...

  render() {
    let me = {...};
    let hobbies = me.hobbies.map(h => <li>{h}</li>);
    return (
      <div className="user-details">
        <h3>{me.name}</h3>
        <p><strong>Age: </strong>{this.getAge(me.birthDate)}</p>
        <p><strong>Gender: </strong><span className={"gender
"+me.gender}>{me.gender === 'male'? String.fromCharCode(9794) :
String.fromCharCode(9792)}</span></p>
        <p><strong>Job title: </strong>{me.jobTitle}</p>
        <p><strong>Hobbies: </strong></p>
        <ul>
          {hobbies}
        </ul>
      </div>);
  }
}

```

3.6.8 (OPTIONAL) EXTRA CHALLENGE

Right now, your app is being rendered on the div with id **root** in your **index.html** file.

Change the id of that div to **hello-react**, and make sure your app renders on the new id instead.

4 COMPONENTS

Agenda



- Interactivity
- How does component state work
- Multiple (child) components
- Transferring Props
- Form Components

Interactivity



- Components are the building blocks of a React application
- What does a component contain?
 - **Constructor**
 - **Render** method
 - **State** mechanism: **state** and **props**
 - **Event handlers**
 - **Lifecycle** methods

Used to make components interactive and reusable

4.1 STATE MACHINES

Components are State Machines



- Data needs to be shown in our render method
 - This makes components reusable!
- How do you keep track of data?
 - **props**: data that got passed in
 - **state**: data that we need to keep track of

What are Props?



- Read-only
- Passed in from calling component
- The "configuration" for your component

```
class ProfileLink extends React.Component{
  render(){
    return (<a href={'https://www.twitter.com/' + this.props.username}>
      {this.props.username}
    </a>);
  }
}

ReactDOM.render(<ProfileLink username="DiedrikDM" />, document.querySelector('div'));
```

4.2 COMPONENT STATE

What is State?



- Stored inside the component
- Can be changed by the component itself
- Shouldn't be changed by parent components
- Allows for interactive components

```
export class ProfileName extends React.Component{
  constructor(){
    super();
    this.state = { profileName : "DiedrikDM" }
  }
  render(){
    return (<p>{this.state.profileName}</p>);
  }
}
```

React-ing on State



- Don't modify state directly (except in constructor), use setState()

```
setState((prevState, props) => nextState, callback)
```

- Merges nextState into this.state and re-renders the component
- After Re-render, optional callback will be called
 - Most of the time, no callback is necessary
- Alternatively, call setState with the part of the state to change

```
setState( { key : value, key2 : value2 } )
```

Using State to Show/Modify Data



JSX

```
export class ProfileName extends React.Component{
  constructor(){
    super();
    this.state = { profileName : "DiedrikDM" }
  }
  changeText = (evt) => {
    let value = evt.target.value;
    this.setState((_, __) => { return { profileName: value } });
  }
  render(){
    return (<form>
      <p>{this.state.profileName}</p>
      <input type="text" onChange={this.changeText}
        value={this.state.profileName} />
    </form>);
  }
}
```

Stateful or Stateless Components?



- Most components should take data from props and render
- Some Components need to respond to
 - User Input
 - Server Request
 - Timer
 - ...
- In other words: most Components are stateless
- Common pattern:
 - Create stateless components to render data
 - Create stateful component above them, to pass state to children
 - Stateful has all interaction logic, stateless render data



What should go in State?

- State contains data changeable by event handlers that may trigger a UI update
- State is very small and JSON-serializable
- Stateful components have minimal representations of state
 - Only store those properties in `this.state`
 - In `render()` function, compute info based on this state



What Shouldn't Go in State?

- `This.state` should contain minimal amount of data and **not**:
 - **Computed data**: precomputing values is unnecessary. Just do computation within `render()`
 - **React components**: build them in `render()` based on props
 - **Duplicated data from props**: use props as direct access to data. Exception is if you need to check previous values.

4.3 INTERACTIVITY

Adding Interactivity

u2u

- To add **interactivity** we need to listen to **events**
- In React this is accomplished by using "onEvent" properties
 - Where Event is replaced with the event to listen to
 - E.g.: onClick, onChange,...
- these events can be used to manipulate the state of a component

Interactivity and Dynamic UIs

u2u

```
JSX
// Simple Example
class LikeButton extends React.Component{
  constructor(){
    this.state = { liked: false};
  }
  handleClick = (event) => {
    this.setState({liked: !this.state.liked});
  }
  render () {
    let text = this.state.liked ? 'like' : 'haven\'t liked';
    return <p onClick={this.handleClick}>
      You {text} this. Click to toggle.
    </p>
  }
}

ReactDOM.render(
  <LikeButton />,
  document.getElementById('example')
);
```

Event Handling and Synthetic Events



- Event Handlers are camelCased props
 - Pass them in with React like in normal HTML
- React lets all events behave identically in IE8+
 - Uses a Synthetic Event System
- Synthetic Event What?
 - React knows the bubble and capture phases following html spec
 - Events passed to event handler are consistent with W3C spec

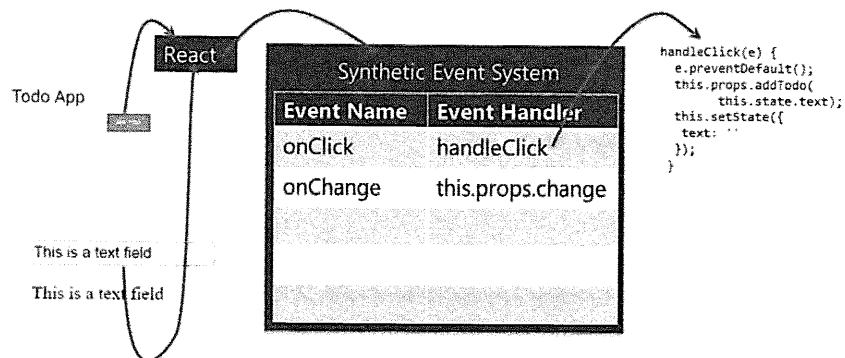
Event Delegation



- Event Delegation:
 - React does not attach event handlers to DOM nodes
- So what happens?
 - At start up: create one event listener at top level for all events
 - When a component is added or removed, its event handlers are added or removed from an internal mapping
 - Event is raised: React dispatches event by using the mapping

Interactivity: Synthetic Event System

U2U



Little Detail

U2U

- If your event handler needs to change the state
 - Make sure to reference the correct "this"

```
class ProfileName extends Component {
  constructor(){
    super();
    this.state = { name : "DiedrikDM" }
  }
  changeText = (evt) => {
    //handle event
  };
  render(){
    return (<p>{this.state.name}</p>);
  }
}
```

```
class ProfileName extends Component{
  constructor(){
    super();
    this.state = { name : "DiedrikDM" }
    this.changeText = this.changeText.bind(this);
  }
  changeText(evt) {
    //handle event
  }
  render(){
    return (<p>{this.state.name}</p>);
  }
}
```

Referencing the Correct Input Field



- How to get the value from a field in an event handler?
 - Event.target
 - Or use a reference

```
export class ProfileName extends React.Component{
  constructor(){
    super();
    this.state = { profileName : "DiedrikDM" }
  }
  changeText = (evt) => {
    let value = evt.target.value;
    this.setState((_, __) => { return { profileName: value } });
  };
  render(){
    return (<input type="text" onChange={this.changeText}
      value={this.state.profileName} />);
  }
}
```

Referencing the Correct Input Field



- References give us access to elements from our render method anywhere inside the component.

```
export default class GameCreate extends Component{
  saveGame = () => {
    let name = this.refs.gamenamebox.value;
    let publisher = this.refs.gamepublisherbox.value;
    this.props.saveGame({name, publisher});
  }

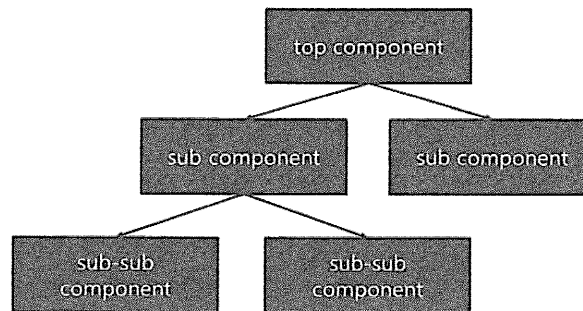
  render(){
    return <div>
      <input ref="gamenamebox" placeholder="Name" />
      <input ref="gamepublisherbox" placeholder="Publisher" />
      <button onClick={this.saveGame}>Create Game</button>
    </div>
  }
}
```

4.4 MULTIPLE COMPONENTS

Multiple Components

u2u

- An application needs multiple components
 - Reusability
 - Separation of Concerns
 - Feels structurally right



Data in a Component Hierarchy

u2u

- The data flow in a Component Hierarchy goes top to bottom
- Passing data to a child component is done with
 - Property binding from the parent
 - Reading out props from the child

Multi-Component Example



```
JSX
class Avatar extends React.Component{
  render() {
    return (
      <div>
        <ProfilePic username={this.props.username} />
        <ProfileLink username={this.props.username} />
      </div>
    );
  }
}

class ProfilePic extends React.Component{
  render() {
    return (
      <img src={'https://twitter.com/'+this.props.username+'/profile_image?size=original'} />
    );
  }
}
```

Children



- It is possible to include Components in a parent Component

```
<Tabs>
  <Pane title={"Pane 1"}>
    <p>Here to cause you pane</p>
  </Pane>
  <Pane title={"Pane 2"}>
    <p>Pane is temporary</p>
  </Pane>
</Tabs>
```

- Parent can read children with `this.props.children`
 - Use `React.Children` utilities to inspect them

Showing Collections



- React updates the DOM with each new render
 - Children also get updated, in the same order as their rendering

- Suppose the following render method

```
render(){
  let pizzas = this.props.childPizzas;
  let listItems = pizzas.map((pizza) =><li data-id={pizza.id}>
                                {this.pizzaToString(pizza)}
                              </li>);

  return <ul>
    {listItems}
  </ul>
}
```

Showing Collections



- Suppose we have two pizzas and we try to remove one

```
// Output before removal
<ul>
  <li data-id="1">Margherita - 8 - Is it spicy? false</li>
  <li data-id="2">Napoletana - 10 - Is it spicy? true</li>
</ul>
// Output after removal
<ul>
  <li data-id="2">Napoletana - 10 - Is it spicy? true</li>
</ul>
```

- Which `<li>` was removed from the HTML?
 - Hint: it wasn't the first list-item

Dynamic Children



- Identify each child by assigning a key

```
render(){
  let pizzas = this.props.childPizzas;
  let listItems = pizzas.map((pizza) =>
    <li key={pizza.id}>{this.pizzaToString(pizza)}</li>);

  return <ul>
    {listItems}
  </ul>
}
```

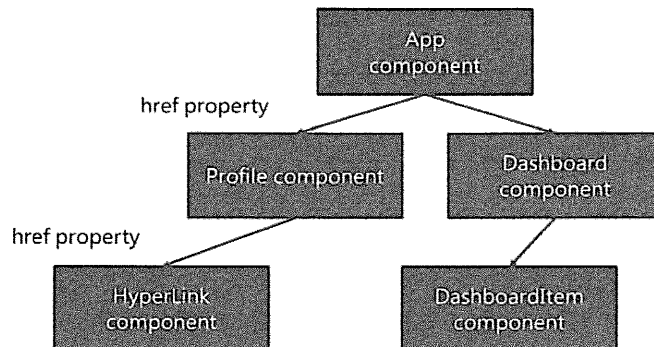
- During updates of keyed children
 - React will ensure child is reordered or destroyed correctly

4.5 TRANSFERRING PROPS

Transferring Props

u2u

- Sometimes we need to pass props from a parent element to child elements



Transferring Props

u2u

- React allows us to copy props automatically
 - Use the spread operator

```
class CheckLink extends React.Component{
  render() {
    // This takes props passed to CheckLink and copies them to <a>
    return <a {...this.props}>{'v '}{this.props.children}</a>;
  }
}

ReactDOM.render(
  <CheckLink href="/checked.html">Click here!</CheckLink>,
  document.getElementById('example')
);
```

Transferring Props



- Sometimes you don't want to pass every property
 - Use the spread operator

```
class FancyCheckbox extends React.Component{

  render() {
    let { checked, ...other } = this.props;
    let fancyClass = checked ? 'FancyChecked' : 'FancyUnchecked';
    // other contains onClick but not the checked attribute
    return (<div {...other} className={fancyClass} />);
  }
}

ReactDOM.render(
  <FancyCheckbox checked={true} onClick={console.log.bind(console)}>
    Hello world!
  </FancyCheckbox>, document.getElementById('example')
);
```


4.6 FORM COMPONENTS

Interactive Properties

u2u

- input, textarea and select fields can be mutated through interactions
- React supports these properties for those elements
 - Value
 - Checked
 - Selected
- Listen to changes with onChange property

Setting the Value

u2u

- Set the value of an Input element
 - Value of rendered element is always the same

```
class MyTextBox extends React.Component{
  render() {
    return <input type="text" value="Hello!" />;
  }
}

ReactDOM.render( <MyTextBox />, document.getElementById('example') );
```

In this example: A textbox appears with no way to change "Hello!"

Changing the Value



- Use the onChange event to capture changes
 - This way a user can change the value in the textbox

```
class MyTextBox extends React.Component{
  constructor() {
    this.state = { value: 'Hello!' };
    this.handleChange = this.handleChange.bind(this);
  }

  handleChange(event) {
    this.setState({ value: event.target.value });
  }

  render() {
    let value = this.state.value;
    return <input type="text" value={value} onChange={this.handleChange} />;
  }
}
```

Setting a Default Value



- Set a changeable default value (without an onChange event)

```
render() {
  return <input type="text" defaultValue="Bye now!" />;
}
```
- This is an input with empty value, listening can still be done with the onChange event
- The defaultChecked property can be used to set an initial check value

4.7 ALTERNATIVE SYNTAX

Stateless Component



JSX

// Stateless components can be written shorter (see next slide)

```
import React from 'react';
```

```
export class UsersList extends React.Component{
  render () {
    let counter = 0;
    if (!this.props.users || !this.props.users.length) { return null;}
    return <ul className="collection">
      {this.props.users.map((user) => {
        return <li key={counter++} className="collection-item">{user}</li>
      })}
    </ul>;
  }
}
```

Alternative Syntax



- Stateless components are often little more than a render method
- We can convert them to functional components
 - Higher signal to noise ratio
 - Potential performance improvements

```
import React from 'react';
```

```
export const UsersList = (users) => {
  let counter = 0;
  if (!users || !users.length) { return null;}

  return <ul className="collection">
    {users.map(function (user) {
      return <li key={counter++} className="collection-item">{user}</li>
    })}
  </ul>;
}
```

4.8 SUMMARY

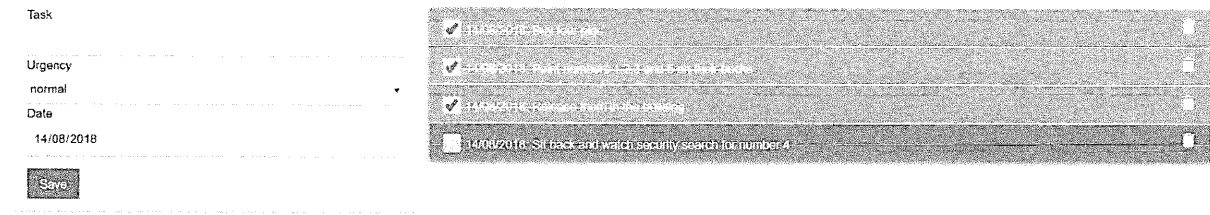
Summary



- How does component state work
 - props comes from parent (immutable)
 - state is your own state (mutable)
- Interactivity
 - Synthetic Event System
- Multiple (child) components
 - props.children
 - keys
- Transferring Props
 - {...Spread}
- Form Components
 - Represent current state, not only initial state
- Functional Components

4.9 LAB – TODO LIST

In this lab you will create a Todo List application with some extras. The end result will look something like this:



4.9.1 BEFORE YOU START

This lab starts from scratch. You will use create-react-app to generate the basic project structure.

4.9.2 CREATING THE PROJECT

1. Open a command line terminal in your source folder, and run the following command in there:

COMMAND LINE

```
create-react-app todolist
----- OR -----
npx create-react-app todolist
```

When this is done, you can open the created project in the IDE of your choice.

4.9.3 GETTING SOME OVERHEAD OUT OF THE WAY

Before you start building the application, let's get some resources to make everything look good.

2. Create a stylesheet under **public/css/styles.css**. add the following rules in there:

STYLES.CSS

```
.done {
  opacity: 0.6;
}
.done label {
  text-decoration: line-through;
}

.w3-btn, .w3-card {
  margin: 1em 0;
}

.material-icons {
  cursor: pointer;
}

.minor {
  color: #fff;
  background-color: #4CAF50;
}

.normal {
  color: #fff;
  background-color: #2196F3;
}

.major {
  color: #fff;
  background-color: #f44336;
}
```

3. Add the following references in the **head** of your **index.html** file:

INDEX.HTML

```
<link rel="stylesheet" href="https://www.w3schools.com/w3css/4/w3.css">
<link rel="stylesheet" href="/css/styles.css">
<link rel="stylesheet"
href="https://fonts.googleapis.com/icon?family=Material+Icons">
```

4. Finally, we'll be using [Moment.js](#) as a helper library for working with dates. Install it with npm.

```
npm i moment
```

4.9.4 CREATING THE COMPONENTS

From what you've seen in the screenshot above, you've identified the following components:

- **App component:** this component will be the top-level component in your application, it will contain the state and behavior, and delegate everything to its sub-components.
- **TodoCreator component:** this component will contain a form for adding new todos.
- **TodoList component:** this will show a list of existing todos.
- **Todo component:** this will be a simple component that shows the details of an existing todo.

5. Start by creating the TodoCreator component in the **src** folder.

TODOCREATOR.JSX

```
import React, { Component } from 'react';
class TodoCreator extends Component {
  render() {
    return (
      <form className="w3-card">
        <div className="w3-container">
          <div className="w3-row">
            <label>Task</label>
            <input className="w3-input" />
          </div>
          <div className="w3-row">
            <label>Urgency</label>
            <select className="w3-select"></select>
          </div>
          <div className="w3-row">
            <label>Date</label>
            <input type="date"/>
          </div>
          <button className="w3-btn w3-blue" type="submit">Save</button>
        </div>
      </form>
    );
  }
}
export default TodoCreator;
```

6. Next, create the Todo component. Create it as a functional component.

TODO.JSX

```
import React from 'react';
export const Todo = (props) => {
  return (
    <li>
      <label>
        <input className="w3-check" type="checkbox" />
        &nbsp;text goes here
      </label>
      <i className="material-icons w3-right">delete</i>
    </li>
  );
}
```

7. Now, create the TodoList component.

TODOLIST.JSX

```
import React, { Component } from 'react';
import { Todo } from './Todo';
class TodoList extends Component {
  render() {
    return (
      <ul className="w3-ul w3-card-4 w3-margin">
        <Todo/>
        <Todo/>
        <Todo/>
      </ul>
    );
  }
}
export default TodoList;
```

8. Finally, add everything into your App component so it gets shown in the application.

APP.JS

```
import React, { Component } from 'react';
import TodoCreator from './TodoCreator';
import TodoList from './TodoList';

class App extends Component {
  render() {
    return (
      <div className="w3-container">
        <div className="w3-container">
          <div className="w3-row">
            <div className="w3-col m4">
              <TodoCreator/>
            </div>
            <div className="w3-col m8">
              <TodoList/>
            </div>
          </div>
        </div>
      </div>
    );
  }
}

export default App;
```

4.9.5 ADDING IN THE STATE

Now that you have a static version of your app, it's time to figure out the state.

- Your App component will contain a list of todo's with a **text**, **urgency**, **date** and **isDone** property. It will pass the todos down to the todoList component, and that one will pass the individual items down to the Todo component.
- Your TodoCreator component will contain a single todo, which is the new one you're creating. Once created, it will pass the new todo to the App component, where it will be added to the list.

9. Add the state for the App component. Pass it to the TodoList component as a prop.

APP.JS

```
import React, { Component } from 'react';
import TodoCreator from './TodoCreator';
import TodoList from './TodoList';

class App extends Component {
  constructor(props) {
    super(props);
    this.state = { todos: [] };
  }

  render() {
    return (
      <div className="w3-container">
        <TodoCreator />
        <TodoList todos={this.state.todos} />
      </div>
    );
  }
}

export default App;
```

10. In the TodoList component, pass along the todos to the Todo component. (don't forget to give the Todos a key!)

TODOLIST.JSX

```
import React, { Component } from 'react';
import { Todo } from './Todo';

class TodoList extends Component {
  render() {
    const todos = this.props.todos.map(
      (todo) => (<Todo key={todo.date.format('X')}+ '-' + todo.text} item={todo} />)
    );
    return (
      <ul className="w3-ul w3-card-4">
        {todos}
      </ul>
    );
  }
}

export default TodoList;
```

11. In the Todo component, show the date and text of the item in the label, give it a class depending on its urgency (you can straight up use the urgency value for this), and give it either the "done" or "open" class depending on its state.

TODO.JSX

```
import React from 'react';
export const Todo = (props) => {
  return (
    <li className={props.item.urgency + ' ' + (props.item.isDone ? "done" :
"open")}>
      <label>
        <input className="w3-check" type="checkbox" />
        &nbsp; {props.item.date.format('DD/MM/YYYY')} {props.item.text}
      </label>
      <i className="material-icons w3-right">delete</i>
    </li>
  );
};
```

12. Now add the state to your TodoCreator component. Create a constant for the urgencies, this will make it easier to create the options for the select.

TODOCREATOR.JSX

```
import React, { Component } from 'react';
import moment from '../node_modules/moment';

const urgency = { MINOR: 'minor', NORMAL: 'normal', MAJOR: 'major' };

class TodoCreator extends Component {
  constructor(props) {
    super(props);
    this.state = { text: "", isDone: false, urgency: urgency.NORMAL, date:
moment().startOf('day') };
  }

  render() {
    const urgencyOptions = Object.values(urgency).map(
      v => <option key={v} value={v}>{v}</option>);
    return (
      <form className="w3-card">
        <div className="w3-container">
          <div className="w3-row">
            <label>Task</label>
            <input className="w3-input" value={this.state.text} />
          </div>
          <div className="w3-row">
            <label>Urgency</label>
            <select className="w3-select" value={this.state.urgency}>
              {urgencyOptions}
            </select>
          </div>
          <div className="w3-row">
            <label>Date</label>
            <input type="date" className="w3-input"
              value={this.state.date.format('YYYY-MM-DD')} />
          </div>
          <button className="w3-btn w3-blue " type="submit">Save</button>
        </div>
      </form>
    );
  }
}

export default TodoCreator;
```

13. Run your application, you should see the TodoCreator, but not be able to change the inputs. (if you want to see the TodoList, you can quickly add some placeholder todos in your App's state)

4.9.6 MAKING THE APPLICATION INTERACTIVE

Now that the structure of your application is in place, you can start adding interactivity. Remember, data flows down, behavior flows up.

- The TodoCreator component will handle 4 events: **onChange** for each of the **inputs**, and **onSubmit** for the **form**.

TODOCREATOR.JSX

```
import React, { Component } from 'react';
import moment from '../node_modules/moment';

const urgency = { MINOR: 'minor', NORMAL: 'normal', MAJOR: 'major' };

class TodoCreator extends Component {
  constructor(props) {
    super(props);
    this.state = { text: '', isDone: false };
  }

  handleSubmit = (e) => {
    e.preventDefault();
    this.props.addTodo(this.state);
    this.setState({ text: '' });
  }

  handleTextChange = (e) => {
    this.setState({ text: e.target.value });
  }

  handleUrgencyChange = (e) => {
    this.setState({ urgency: e.target.value });
  }

  handleDateChange = (e) => {
    this.setState({ date: moment(e.target.value) });
  }

  render() {
    return (
      <form className="w3-card">
        <div className="w3-container">
          <div className="w3-row">
            <label>Task</label>
            <input className="w3-input" value={this.state.text}
              onChange={this.handleTextChange} />
          </div>
          <div className="w3-row">
            <label>Urgency</label>
            <select className="w3-select" value={this.state.urgency}
              onChange={this.handleUrgencyChange} >
              {urgencyOptions}
            </select>
          </div>
          <div className="w3-row">
            <label>Date</label>
            <input type="date" className="w3-input"
              value={this.state.date.format('YYYY-MM-DD')}
              onChange={this.handleDateChange} />
          </div>
          <button className="w3-btn w3-blue" type="submit">Save</button>
        </div>
      </form>
    );
  }
}

export default TodoCreator;
```

- In the `handleSubmit` method, you're referencing a prop called **addTodo**. Add a method for it to your App component. Since you're using the todo's text and date as a key for the list, make sure you can't add duplicate todos. Also order your todos chronologically

APP.JS

```

import React, { Component } from 'react';
import TodoCreator from './TodoCreator';
import TodoList from './TodoList';

class App extends Component {
  constructor(props) {
    super(props);
    this.state = { todos: [] };
  }

  addTodo = (item) => {
    if (this.state.todos.find((i) => i.text === item.text
      && i.date === item.date)) {
      return;
    }
    this.setState({ todos: [...this.state.todos, item].sort((a, b) =>
      a.date.isAfter(b.date)) });
  }

  render() {
    return (
      <div className="w3-container">
        <div className="w3-row">
          <div className="w3-col m4">
            <TodoCreator addTodo={this.addTodo} />
          </div>
          <div className="w3-col m8">
            <TodoList todos={this.state.todos} />
          </div>
        </div>
      </div>
    );
  }
}

export default App;

```

16. Looks like we forgot something. There isn't any validation on your todoCreator component. You'll want to make sure of 2 things:

- The text for a new todo can't be empty.
- You can't create todos for past dates.

17. Add a validation function that checks these things. Store possible error messages in the component state, and make the function return a Boolean indicating the validity of the form. Call the function when submitting the form, and show some visual feedback if the form isn't valid.

TODOCREATOR.JSX

```

import React, { Component } from 'react';
import moment from '../node_modules/moment';
const urgency = { MINOR: 'minor', NORMAL: 'normal', MAJOR: 'major' };

class TodoCreator extends Component {
  constructor(props) {
    super(props);
    this.state = { text: '', isDone: false, urgency: urgency.NORMAL, date:
moment().startOf('day'), errors: {} };
  }

  handleSubmit = (e) => {
    e.preventDefault();
    if (this.validate()) {
      const { errors, ...todo } = this.state;
      this.props.addTodo(todo);
      this.setState({ text: '', urgency: urgency.NORMAL, date:
moment().startOf('day'), errors: {} });
    }
  }

  handleTextChange = (e) => {
    this.setState({ text: e.target.value });
  }

  handleDateChange = (e) => {
    this.setState({ date: moment(e.target.value) });
  }

  handleUrgencyChange = (e) => {
    this.setState({ urgency: e.target.value });
  }

  validate = () => {
    let valid = true, errors = {};
    if (this.state.text.trim() === '') {
      errors = { ...errors, text: 'text cannot be empty' };
      valid = false;
    }
    let today = moment().startOf('day');
    if (this.state.date.isBefore(today)) {
      errors = { ...errors, date: 'date cannot be in the past' };
      valid = false;
    }
    this.setState({ errors: errors });
    return valid;
  }

  render() {
    const urgencyOptions = Object.values(urgency).map(v => <option key={v}
value={v}>{v}</option>);
    return (
      <form className="w3-card" onSubmit={this.handleSubmit}>
        <div className="w3-container">
          <div className="w3-row">
            <label>Task</label>
            <label className="w3-text-red w3-
right">{this.state.errors.text}</label>
            <input className={"w3-input " + (this.state.errors.text ? 'w3-
border-red' : '')} onChange={this.handleTextChange} value={this.state.text} />
          </div>
          <div className="w3-row">
            <label>Urgency</label>
            <select className="w3-select" value={this.state.urgency}
onChange={this.handleUrgencyChange}>
              {urgencyOptions}
            </select>
          </div>
          <div className="w3-row">
            <label>Date</label>
            <label className="w3-text-red w3-
right">{this.state.errors.date}</label>
            <input type="date" className={"w3-input " +
(this.state.errors.date ? 'w3-border-red' : '')}
onChange={this.handleDateChange} value={this.state.date.format('YYYY-MM-DD')}
/>
          </div>
          <button className="w3-btn w3-blue" type="submit">Save</button>
        </div>
      </form>
    );
  }
}

```

```

    }
  }
};

export default TodoCreator;

```

18. Next up is marking todos as done, you'll use the checkbox in the Todo component for this. Since the Todo component is a functional component, you'll have to get creative.

TODO.JSX

```

import React from 'react';

export const Todo = (props) => {
  return (
    <li className={props.item.urgency + ' ' + (props.item.isDone ? "done" :
    "open")}>
      <label>
        <input className="w3-check" type="checkbox" onChange={() =>
        props.toggleTodo(props.item)} checked={props.item.isDone} />
        &nbsp;{props.item.date.format('DD/MM/YYYY')}: {props.item.text}
      </label>
      <i className="material-icons w3-right">delete</i>
    </li>
  );
}

```

19. Let the event flow up until you get to the App component, and handle it there.

TODO.LIST.JSX

```
import React, { Component } from 'react';
import { Todo } from './Todo';

class TodoList extends Component {
  render() {
    const todos = this.props.todos.map((todo) => (<Todo
      key={todo.date.format('x')+ '-' + todo.text} item={todo}
      toggleTodo={this.props.toggleTodo} />))
    return (
      <ul className="w3-ul w3-card-4">
        {todos}
      </ul>
    );
  }
}

export default TodoList;
```

APP.JS

```
...

class App extends Component {
  ...

  toggleTodo = (item) => {
    const index = this.state.todos.indexOf(item);
    let newItem = {...item, isDone: !item.isDone};
    const newTodos = this.state.todos.slice(0, index)
      .concat([newItem])
      .concat(this.state.todos.slice(index + 1));
    this.setState({ todos: newTodos });
  }

  render() {
    return (
      <div className="w3-container">
        <div className="w3-row">
          <div className="w3-col m4">
            <TodoCreator addTodo={this.addTodo} />
          </div>
          <div className="w3-col m8">
            <TodoList todos={this.state.todos} toggleTodo={this.toggleTodo} />
          </div>
        </div>
      </div>
    );
  }
}

export default App;
```

Run your application and see it working.

4.9.7 (OPTIONAL) EXTRA CHALLENGE: MAKE THE DELETE BUTTON WORK

Given the skills you've honed during this exercise, try making the delete button on the Todo component work.

5 COMPONENT LIFECYCLE AND PERFORMANCE

Agenda

The logo for u2u, consisting of the letters 'u2u' in a white, sans-serif font inside a dark square.

- Component Lifecycle
- Lifecycle Methods
- Loading Data
- Virtual DOM
- Tweaking Performance
- PureComponent

5.1 COMPONENT LIFECYCLE

Component Lifecycle



- Three main parts
 - Mounting (insert into DOM)
 - Updating (re-render and check if DOM should be updated)
 - Unmounting (remove from DOM)
- Hook into the lifecycle with methods
 - "Will" methods: before something happens
 - "Did" methods: after something happened

Lifecycle Methods



- Mounting:
 - constructor, `getDerivedStateFromProps`, `componentDidMount`
- Updating:
 - `getDerivedStateFromProps`, `shouldComponentUpdate`, `getSnapshotBeforeUpdate`, `componentDidUpdate`
- Unmounting
 - `componentWillUnmount`, `componentDidCatch`
- UNSAFE_ methods exist too
 - Too many potential pitfalls to be safely used for async rendering. (deprecated in React 17)

Loading Data



- When should you load data into your component?
 - 2 options: **constructor** or **componentDidMount**
- No async code in constructor!
- Load data in the componentDidMount hook
- How should you load the data?
 - Use fetch or a third party library: Axios, jQuery Ajax, ...

```
componentDidMount() {  
  fetch(`https://randomuser.me/api/?results=5`).  
    then(response => response.json()).then(data => {  
      let tempUsers = data.results;  
      this.setState({users: tempUsers});  
    });  
}
```

5.2 ADVANCED PERFORMANCE

Virtual DOM



- What is the Virtual DOM?
 - Very fast in-memory representation of the DOM
 - To make sure React does not talk to the DOM directly
- render() methods return a description which is compared to the in-memory representation
- If necessary, React allows Virtual DOM bypasses

Tweaking the Performance



- When props or state changes
 - React will look if it needs to update by calling shouldComponentUpdate
 - React will check old with new Virtual DOM
 - Actual DOM will be reconciled only if different Virtual DOM
- shouldComponentUpdate()
 - Component lifecycle event triggered before re-rendering process
 - By default (when not implemented) returns true

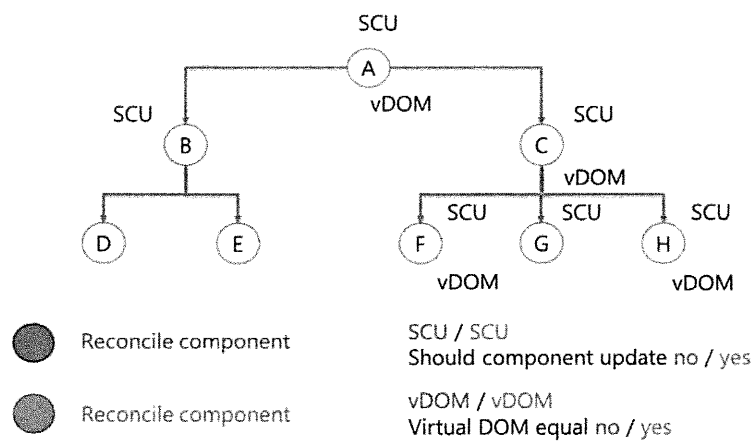
shouldComponentUpdate()



- Method can be short circuited
 - To provide faster change detection for the application
- React calls this function a lot
 - Needs to be very quick!
- Signature of shouldComponentUpdate

`shouldComponentUpdate: (nextProps, nextState) => boolean`

shouldComponentUpdate In Action



How To Implement sCU()?



- A class with just a string as value
 - shouldComponentUpdate is pretty simple

```
class MyComponent extends React.Component{  
  render: function() {  
    return <div>{this.props.value}</div>;  
  }  
}
```

sCU for String



- Dealing with simple properties is easy

```
class MyComponent extends React.Component{  
  shouldComponentUpdate: function(nextProps, nextState) {  
    return this.props.value !== nextProps.value;  
  }  
  render: function() {  
    return <div>{this.props.value}</div>;  
  }  
}
```

sCU for Complex Types



- What happens if sCU should be used on complex objects
 - Data structures that are mutable

```
class MyComponent extends React.Component{  
  
  // Is this a correct implementation for sCU?  
  shouldComponentUpdate: function(nextProps, nextState) {  
    return this.props.value !== nextProps.value;  
  }  
  
  render() {  
    return <div>{this.props.value.foo}</div>;  
  }  
}
```

Implementing sCU for Objects



- This will create a bad shouldComponentUpdate()

```
// assume this.props.value is { foo: 'bar' }  
// assume nextProps.value is { foo: 'bar' },  
// but references are different  
this.props.value !== nextProps.value; // will always be true
```

- Alternatively: deep comparison

```
shouldComponentUpdate: function(nextProps, nextState) {  
  return this.props.value.foo !== nextProps.value.foo;  
}
```

sCU for Complex Types



- Deep checking values

```
class MyComponent extends React.Component{

  // This will now give correct results!
  shouldComponentUpdate: function(nextProps, nextState) {
    return this.props.value.foo !== nextProps.value.foo;
  }

  render() {
    return <div>{this.props.value.foo}</div>;
  }
}
```

Complex Types can be Complex!



- shouldComponentUpdate methods are not really maintainable nor do they scale well like this

```
class MyComponent extends React.Component{

  // This method becomes unmanageable
  shouldComponentUpdate: function(nextProps, nextState) {
    return this.props.value.foo !== nextProps.value.foo ||
           this.props.value.bar !== nextProps.value.bar ||
           this.props.value.beers[0] !== nextProps.value.beers[0] ||
           this.props.value.beers[2] !== nextProps.value.beers[2];
    // ...
  }

  render() {
    return <div> ... </div>;
  }
}
```

Immutable-js To the Rescue



- JavaScript collections library
 - Provides immutable persistent collections via structural sharing
- Immutable
 - Once created, collection cannot be altered
- Persistent
 - New collections can be created from previous + mutation
 - Old collection is still valid afterwards
- Structural Sharing
 - New collection re-uses as much of the old structure as possible

Immutable Change Tracking



- Cheap change tracking
 - Changes will result in new objects

- Pure JavaScript

```
var x = { foo: "bar" };  
var y = x;  
y.foo = "baz";  
x === y; // true
```

- Vs Immutable-js

```
var SomeRecord = Immutable.Record({ foo: null });  
var x = new SomeRecord({ foo: "bar"});  
var y = x.set("foo", "baz");  
x === y; // false
```


Immutable Data Structures



- Provides cheap and less verbose way of change tracking
 - Good for `shouldComponentUpdate`
- If props and state are modelled with `immutable-js`
 - Use `React.PureComponent` for the performance boost

5.3 PURECOMPONENT

PureComponent



- `React.PureComponent` is an alternative to `React.Component`
 - For "pure" components:
 - This means the output should rely only on state and props
- Can give a performance boost
- `PureComponent` implements `shouldComponentUpdate()`
 - Does shallow comparison
 - Only works correctly if child elements are "pure" as well

```
class MyComponent extends React.PureComponent{  
  render() {  
    return <div> ... </div>;  
  }  
}
```

6 STRUCTURING APPLICATIONS TOWARDS COMPONENTS

Agenda



- Start with a Mock
- Splitting a Mock into Components
- Building a static version in React
- Figuring out the UI state
- Where should State live?
- Creating Inverse Data Flow

6.1 THINKING IN REACT

Thinking in React

u2u

- React has quick way to create big, fast web apps
 - React makes you think about apps while building them
- If you follow the next couple of steps
 - Creating a new application in React will come naturally

Start With a Mock

u2u

- Let a designer make a design
 - Fill it with mock data

Search...

☐ Only show products in stock

Name	Price
Sporting Goods	
Football	\$49.99
Baseball	\$9.99
Basketball	\$29.99
Electronics	
iPod Touch	\$99.99
iPhone 5	\$399.99
Nexus 7	\$199.99

```
[
  {
    category: "Sporting Goods",
    price: "$49.99", stocked: true,
    name: "Football" },
  {
    category: "Sporting Goods",
    price: "$9.99", stocked: true,
    name: "Baseball" },
  {
    category: "Sporting Goods",
    price: "$29.99", stocked: false,
    name: "Basketball" },
];
```

Breaking UI into Component Hierarchy



- Draw some boxes around the design
 - Identify the pieces of data that go together, and their hierarchy
 - Could already be created by your designer!

Search	
Only show products in stock	
Name	Price
Sporting Goods	
Football	\$49.99
Baseball	\$9.99
Basketball	\$29.99
Electronics	
iPod Touch	\$99.99
iPhone 5	\$399.99
Nexus 7	\$199.99

What's a Component?



- Programmer's logic
 - **Single Responsibility Principle**: component should only do one thing
 - If component grows: decompose into smaller components
- Component could be a model mapped from JSON
 - Makes it easy to create components

Try it On the Example



- Five components can be found on the example:
 - **FilterableProductTable** (orange): contains everything
 - **SearchBar** (blue): receives input
 - **ProductTable** (green): displays and filters
 - **ProductCategoryRow** (turquoise): displays heading
 - **ProductRow** (Red): displays row
- ProductTable has a header for Name and Price labels
 - Can be separate component or not

Search	
Only show products in stock	
Name	Price
Sporting Goods	
Football	\$49.99
Baseball	\$9.99
Basketball	\$29.99
Electronics	
iPod Touch	\$99.99
iPhone 5	\$399.99
Nexus 7	\$199.99

Try it On the Example



- Next is arranging them into hierarchy
 - Just look at the drawing you made
- FilterableProductTable
 - SearchBar
 - ProductTable
 - ProductCategoryRow
 - ProductRow

Search	
Only show products in stock	
Name	Price
Sporting Goods	
Football	\$49.99
Baseball	\$9.99
Basketball	\$29.99
Electronics	
iPod Touch	\$99.99
iPhone 5	\$399.99
Nexus 7	\$199.99

Building a Static Version



- With component hierarchy
 - Time to implement app
- Build Components taking data and rendering UI
 - Without Interactivity
- Building static version is lots of typing and less thinking
 - Interactive version is vice versa

Building Components



- Build components that reuse other components
 - Pass data from parent to child using props
- Don't use state at all for the static version
 - Reserved only for interactivity
- Components are available after this step
 - Only render() method is done
 - Top component takes data as prop

Identifying Minimal State



- Making UI interactive
 - Needs triggers for data – React does this with State
- Think of the minimal set of state for the app
 - DRY!
- Three questions:
 - Is it passed from parent to child with props? → Yes: means no state
 - Does it change over time? → No: means no state
 - Can I compute it based on other state or props? → Yes: means no state

State in Example



- Pieces of data in example are:
 - List of products
 - Search text from user
 - Value of checkbox
 - Filtered list of products
- List is passed in as props: no state
- Search text and checkbox appear to be state: can change
- Filtered list: no state, can be computed

Where Should State Live?



- Once the state is identified, determine where it should be kept
 - React does one-way data flow down component hierarchy
- For each piece of state in the app
 - Identify all components that are rendered based on this state
 - Find common owner component
 - Common owner or other higher up should contain state
 - Doesn't make sense? Add extra component for state

State Lives Here in the Example



- ProductTable needs to filter based on state
 - SearchBar needs to display search text and checked state
 - Common owner is FilterableProductTable
- ➔ So let's create it here

Adding Inverse Data Flow - Interactivity



- For now an app exists rendering correctly
 - But only props and state flowing down hierarchy
- Sometimes we need data flowing the other way
 - Like form components deep in the hierarchy updating state
- With React the data flow becomes explicit
 - To better understand how the program works
 - More typing however

Inverse Data Flow in the Example



- Whenever user changes form
 - State should be updated
- Components should only update their own state!
 - So we need a callback from `FilterableProductTable` to `SearchBar`
- Seems complex
 - But this makes the data flow really explicit (and more clear!)

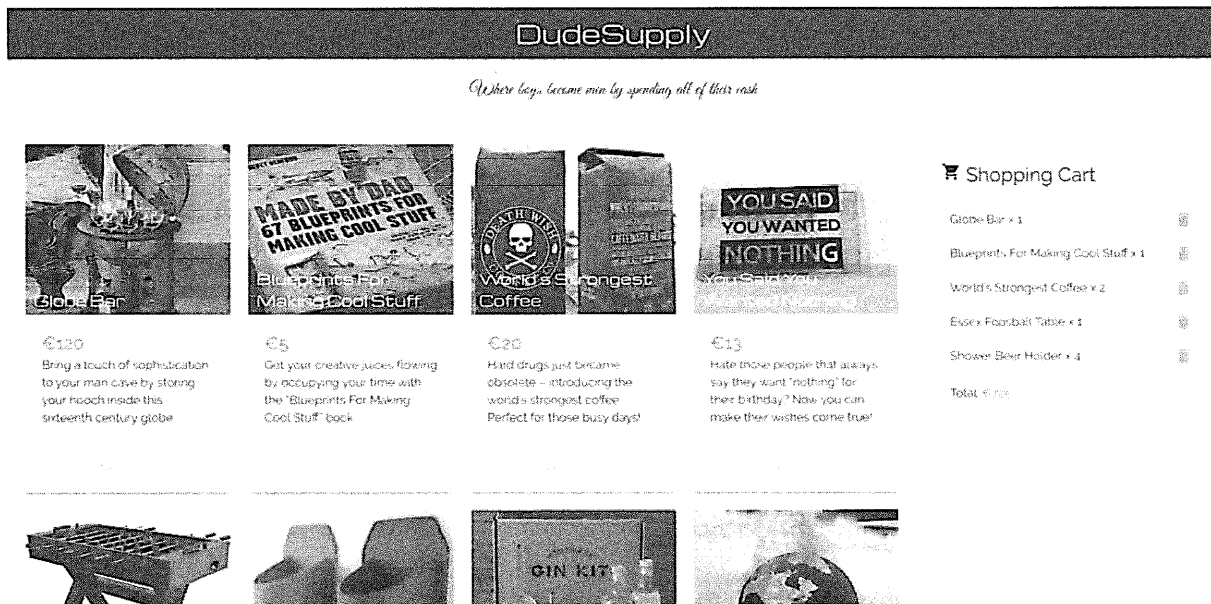
Summary



- Start with a Mock
- Splitting a Mock into Components
- Building a static version in React
- Figuring out the UI state
- Where should State live?
- Creating Inverse Data Flow

6.2 LAB – CREATING A WEBSHOP WITH REACT

In this lab you will create a webshop called DudeSupply. The end result will look something like this:



6.2.1 BEFORE YOU START

This lab starts from starter files found in the "Webshop Starter" folder. Ask your trainer.

In the starter files you will find (among others):

- **public/index.html**: a mock of what your application should look like when it's done.
- **src/App.js**: the main component for your application.
- **src/data.js**: a list of products that you'll use to push into your application later on.

Also note that we added some images, the Materialize css library, and some fonts that we'll be using.

6.2.2 CHECK OUT YOUR MOCK

20. Start by taking a look at your mock. To do this, first **install the dependencies through npm**.

COMMAND LINE

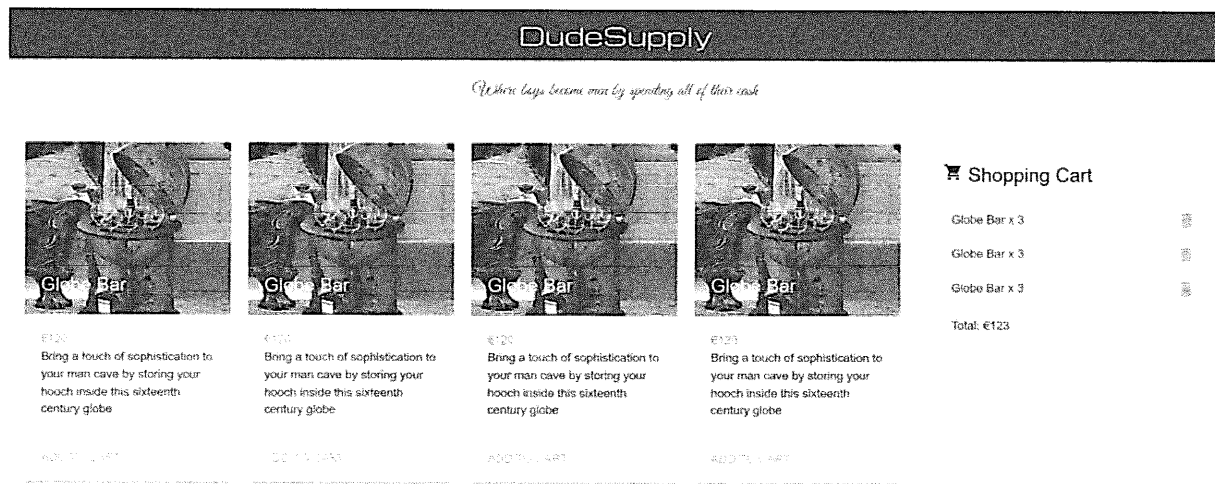
```
npm install
```

21. Once that's done, you can **check out the mock** by starting your application.

COMMAND LINE

```
npm start
```

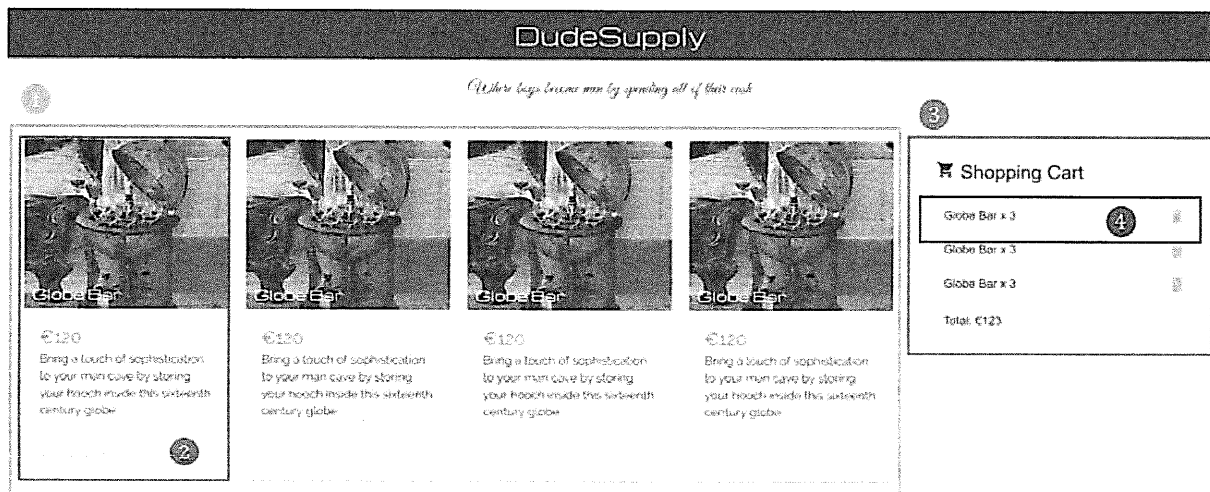
You should see something like this:



6.2.3 SPLITTING UP THE MOCK INTO COMPONENTS

22. **Identify the possible components** inside your mock. Do this by **drawing boxes** around the mock (think of the **Single Responsibility Principle**!)

You might come up with something like this:



- ① **Product list:** a container where you'll show the products you're selling.
- ② **Product detail:** a card where you'll show the details for each product, and an order button.
- ③ **Shopping cart:** The list of items the user wants to buy.
- ④ **Order line:** a line for each order item containing the product, amount, and a button to remove an item.

23. **Arrange** the components you've identified into a **hierarchy**:

- Product list
 - Product detail
- Shopping cart

- Order line

6.2.4 BUILDING A STATIC VERSION

6.2.4.1 CREATING THE CLASSES

24. Create Classes in the src folder for each of your components:

PRODUCTLIST.JSX

```
import React, { Component } from 'react';

class ProductList extends Component {
  render() {
    return (
      <React.Fragment></React.Fragment>
    );
  }
}

export default ProductList;
```

PRODUCTDETAIL.JSX

```
import React, { Component } from 'react';

class ProductDetail extends Component {
  render() {
    return (
      <React.Fragment></React.Fragment>
    );
  }
}

export default ProductDetail;
```

SHOPPINGCART.JSX

```
import React, { Component } from 'react';

class ShoppingCart extends Component {
  render() {
    return (
      <React.Fragment></React.Fragment>
    );
  }
}

export default ShoppingCart;
```

ORDERLINE.JSX

```
import React, { Component } from 'react';

class OrderLine extends Component {
  render() {
    return (
      <React.Fragment></React.Fragment>
    );
  }
}

export default OrderLine;
```

6.2.4.2 ADDING IN THE TEMPLATES

25. Let's **start with the inner components** and work our way up. You can start by **extracting the ProductDetail html** from the **mock** and **placing** it inside the **render** method of your component. (don't forget to rename all of the **class** attributes to **className**!)

PRODUCTDETAIL.JSX

```
import React, { Component } from 'react';

class ProductDetail extends Component {
  render() {
    return (
      <div className="col s12 m3">
        <div className="card">
          <div className="card-image">
            
            <span className="card-title">Globe Bar</span>
          </div>
          <div className="card-content">
            <span className="productPrice light-green-text">&euro;120&nbsp;</span>
            <p>Bring a touch of sophistication to your man cave by storing your hooch inside this sixteenth century globe</p>
          </div>
          <div className="card-action">
            <a>Add To Cart</a>
          </div>
        </div>
      </div>
    );
  }
}

export default ProductDetail;
```

26. Now **extract the template** for the surrounding **ProductList** component. You will be rendering a list of productDetails in here, so for now just mock this by **adding a couple of ProductDetails directly in the template**. (don't forget to **import** the **ProductDetail** component, otherwise you can't use it here.)

PRODUCTLIST.JSX

```
import React, { Component } from 'react';
import ProductDetail from './ProductDetail';

class ProductList extends Component {
  render() {
    return (
      <div className="col s12 m9">
        <ProductDetail/>
        <ProductDetail/>
        <ProductDetail/>
      </div>
    );
  }
}

export default ProductList;
```

27. **Repeat** this process for the **OrderLine** and **ShoppingCart** components.

ORDERLINE.JSX

```
import React, { Component } from 'react';

class OrderLine extends Component {
  render() {
    return (
      <li className="collection-item">Globe Bar x 3<a className="secondary-
content orange-text text-accent-2"><i className="material-
icons">delete</i></a></li>
    );
  }
}

export default OrderLine;
```

SHOPPINGCART.JSX

```
import React, { Component } from 'react';
import OrderLine from './OrderLine';

class ShoppingCart extends Component {
  render() {
    return (
      <div className="col s12 m3">
        <ul className="collection with-header">
          <li className="collection-header"><h5><i className="material-
icons">shopping_cart</i> Shopping Cart</h5></li>
          <OrderLine/>
          <OrderLine/>
          <OrderLine/>
          <li className="collection-item"><h6>Total: <span
className="cartTotal">&euro;123</span></h6></li>
        </ul>
      </div>
    );
  }
}

export default ShoppingCart;
```

28. Now that your components are created, you can insert them into your app component

APP.JS

```
import React, { Component } from 'react';
import logo from './logo.svg';
import './App.css';
import ProductList from './ProductList';
import ShoppingCart from './ShoppingCart';

class App extends Component {
  render() {
    return (
      <div className="row">
        <ProductList/>
        <ShoppingCart/>
      </div>
    );
  }
}

export default App;
```

29. Swap out the mock for your app component, and inspect the result.

INDEX.JS

```
import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import App from './App';
import registerServiceWorker from './registerServiceWorker';

ReactDOM.render(<App />, document.getElementById('root'));
registerServiceWorker();
```

INDEX.HTML

```
<!DOCTYPE html>
<html lang="en">

<head>
</head>

<body>
  <noscript>
    You need to enable JavaScript to run this app.
  </noscript>

  <nav class="blue-grey darken-4">
    <div class="nav-wrapper">
      <a href="#" class="brand-logo center">DudeSupply</a>
    </div>
  </nav>

  <div class="row">
    <p class="tagline center">where boys become men by spending all of their
    cash</p>
  </div>
  <div id="root"></div>
</body>
</html>
```

6.2.5 ADDING IN STATE

Now that you've got the components in place, it's time to **identify what state** you're going to need and **where it's going to live**.

6.2.5.1 DEFINING THE SHAPE OF THE STATE

30. As far as the **products** themselves go, the work has been done for you. If you inspect the **data.js** file, you can see there's a list of products that looks like this:

DATA.JS

```
products: [
  {
    name: 'Globe Bar',
    description: 'Bring a touch of sophistication ...',
    price: 120,
    productImage: 'globe-bar.jpg'
  },
  ...
]
```

Now you'll need to identify the state for your **shopping cart**.

Looking at the cart, you find the following:

- **OrderLine** needs a **product** and an **amount**: you aren't going to copy the entire product in here so you'll need to select a key for it. For simplicity's sake, let's use the **product name** here, since that's the only thing visible on an orderLine.
- **ShoppingCart** needs a **total price**: this can be easily calculated if we know the amount and unit price for each orderLine, so let's add a **unitPrice** to our **OrderLine**.

You'll end up with the following shape (**you don't have to type this anywhere**):

```
orderItems = [
  {
    name: 'Globe Bar',
    amount : 1,
    unitPrice: 120
  },
  ...
]
```

6.2.5.2 WHERE DOES THE STATE LIVE?

You identified that the ProductList will take in the products array as props, and the same can be said for the ShoppingCart and orderItems, so the only logical place to store the state is in the App component.

31. **Add the state** to your app component, and **pass the correct props** to your child components.

APP.JS

```
import React, { Component } from 'react';
import logo from './logo.svg';
import './App.css';
import ProductList from './ProductList';
import ShoppingCart from './ShoppingCart';
import { data } from './data.js';

class App extends Component {
  state = { products: [], orderItems: [] };

  componentDidMount() {
    this.setState({
      products: data.products
    });
  }

  render() {
    return (
      <div className="row">
        <ProductList products={this.state.products} />
        <ShoppingCart items={this.state.orderItems} />
      </div>
    );
  }
}

export default App;
```

Note that we're using the **componentDidMount** lifecycle hook here, that's because normally, **loading data is an async operation**, and there should be **no async operations in the constructor**.

PRODUCTLIST.JSX

```
import React, { Component } from 'react';
import ProductDetail from './ProductDetail';

class ProductList extends Component {
  render() {
    const products = this.props.products.map((p) => (<ProductDetail
    key={p.name} product={p} />));
    return (
      <div className="col s12 m9">
        {products}
      </div>
    );
  }
}

export default ProductList;
```

PRODUCTDETAIL.JSX

```
import React, { Component } from 'react';
import './ProductDetail.css';

class ProductDetail extends Component {
  render() {
    const product = this.props.product;
    return (
      <div className="col s12 m3">
        <div className="card">
          <div className="card-image">
            <img src={'/img/'+product.productImage} />
            <span className="card-title">{product.name}</span>
          </div>
          <div className="card-content">
            <span className="productPrice light-green-text">
              &euro;{product.price}&nbsp;</span>
            <p>{product.description}</p>
          </div>
          <div className="card-action">
            <a>Add To Cart</a>
          </div>
        </div>
      </div>
    );
  }
}

export default ProductDetail;
```

32. The shopping cart will also need some extra styling:

SHOPPINGCART.CSS

```
.cartTotal{
  color: #8bc34a !important;
  font-weight: bold;
}
h5, p, li{
  font-family: 'Raleway', sans-serif;
```

33. Don't forget to create the function that calculates your total price.

SHOPPINGCART.JSX

```

import React, { Component } from 'react';
import OrderLine from './OrderLine';
import './ShoppingCart.css';

class ShoppingCart extends Component {
  getTotalPrice = () => {
    let total = 0;
    this.props.items.forEach((i) => {
      total += i.unitPrice * i.amount;
    });
    return total;
  }

  render() {
    const lines = this.props.items.map((i) => (<OrderLine key={i.name} item={i} />))
    return (
      <div className="col s12 m3">
        <ul className="collection with-header">
          <li className="collection-header"><h5><i class="material-icons">shopping_cart</i> Shopping Cart</h5></li>
          {lines}
          <li className="collection-item"><h6>Total: <span
            className="cartTotal">&euro;{this.getTotalPrice()}</span></h6></li>
        </ul>
      </div>
    );
  }
}

export default ShoppingCart;

```

34. All that's left is the OrderLine component.

ORDERLINE.JSX

```

import React, { Component } from 'react';

class OrderLine extends Component {
  render() {
    return (
      <li className="collection-item">{this.props.item.name} &times;
        {this.props.item.amount}<a className="secondary-content orange-text text-accent-2"><i class="material-icons">delete</i></a></li>
    );
  }
}

export default OrderLine;

```

6.2.6 CREATING THE INVERSE DATA FLOW A.K.A. ADDING INTERACTIVITY

It's time to **bring your application to life**. Think about **what actions** the user can perform on your application, **how they'll trigger them**, and **what needs to happen** when they perform said actions. Remember that **components should only update their own state**!

35. When a user clicks the **add to cart** button in the ProductDetail component, you need to **add a new line** into your orderItems (or **increment the amount** if the product is already in there), which lives in the **App** component.

The data will flow as follows: **ProductDetail → ProductList → App**.

PRODUCTDETAIL.JSX

```
import React, { Component } from 'react';
import './ProductDetail.css';

class ProductDetail extends Component {
  render() {
    const product = this.props.product;
    return (
      <div className="card-action">
        <a onClick={()=>this.props.addToCart(product)}>Add To Cart</a>
      </div>
    );
  }
}

export default ProductDetail;
```

PRODUCTLIST.JSX

```
import React, { Component } from 'react';
import ProductDetail from './ProductDetail';

class ProductList extends Component {
  render() {
    const products = this.props.products.map((p) => (<ProductDetail
key={p.name} product={p} addToCart={this.props.addProductToCart} />));
    return (
      <div className="col s12 m9">
        {products}
      </div>
    );
  }
}

export default ProductList;
```

APP.JS

```
class App extends Component {
  handleAdd = (product) => {
    let newItem = { name: product.name, amount: 1, unitPrice: product.price };
    let newOrderItems;

    let item = this.state.orderItems.find(e1 => e1.name === product.name);
    if (item) {
      newItem.amount = item.amount + 1; //increment amount
      newOrderItems = this.state.orderItems.slice(); //shallow copy of array
      let itemIndex = this.state.orderItems.indexOf(item);
      newOrderItems[itemIndex] = newItem; //replace item with newItem
    } else {
      newOrderItems = this.state.orderItems.concat(newItem);
    }
    this.setState({
      orderItems: newOrderItems
    });
  };

  render() {
    return (
      <div className="row">
        <ProductList products={this.state.products}
          addProductToCart={this.handleAdd} />
        <ShoppingCart items={this.state.orderItems} />
      </div>
    );
  }
}
```

```
export default App;
```

36. When a user clicks the **trash icon** on an order line, You'll **decrement the amount** of that product in the orderItems, and **when it drops to 0, remove the line**.

The data will flow as follows: **OrderLine → ShoppingCart → App**.

ORDERLINE.JSX

```
import React, { Component } from 'react';

class OrderLine extends Component {
  render() {
    return (
      <li className="collection-item">{this.props.item.name} &times;
      {this.props.item.amount}<a className="secondary-content orange-text text-
      accent-2" onClick={() => this.props.removeProduct(this.props.item.name)}><i
      className="material-icons">delete</i></a></li>
    );
  }
}

export default OrderLine;
```

SHOPPINGCART.JSX

```
...

class ShoppingCart extends Component {
  ...

  render() {
    const lines = this.props.items.map((i) => (<OrderLine key={i.name} item={i}
    removeProduct={this.props.removeProductFromCart} />))
    return (
      ...
    );
  }
}

export default ShoppingCart;
```

APP.JS

```
...
class App extends Component {
  ...
  handleRemove = (itemName) => {
    let newItem, newOrderItems;

    let item = this.state.orderItems.find(e1 => e1.name === itemName);

    if (!item) {
      return; //product not found
    }

    newItem = {...item, amount: item.amount - 1}; //decrement amount

    if (newItem.amount > 0) {
      newOrderItems = this.state.orderItems.slice(); //shallow copy of array
      let itemIndex = this.state.orderItems.indexOf(item);
      newOrderItems[itemIndex] = newItem; //replace item with newItem
    } else {
      newOrderItems = this.state.orderItems.filter(e1 => e1 !==
      item); //amount === 0, remove from list
    }
    //remove below here
    this.setState({
      orderItems: newOrderItems
    });
  };

  render() {
    return (
      <div className="row">
        <ProductList products={this.state.products}
        addProductToCart={this.handleAdd} />
        <ShoppingCart items={this.state.orderItems}
        removeProductFromCart={this.handleRemove}/>
      </div>
    );
  }
}
```



```
    } } );  
  }  
  export default App;
```

6.2.7 (OPTIONAL) EXTRA CHALLENGE: DON'T SHOW AN EMPTY CART

Make sure the shopping cart isn't displayed when it's empty.

7 SPA WITH REACT

Agenda



- Towards Single Page Applications
- Introducing React-Router
- Nested Routes
- Advanced Usage

7.1 TOWARDS SINGLE PAGE APPLICATIONS

Towards Single Page Applications



- People are used to the responsiveness of native applications

- Multipage applications have slow page reloads
 - This breaks the fluid user experience

- SPAs prefetch and cache a lot of resources
 - Mimick the speed of native apps
 - Requires a way to do client-side routing
 - Server is only used for data

7.2 INTRODUCING REACT ROUTER

Introducing React Router



- Powerful routing library
 - Built on top of React
 - Helps adding new screens and flows quickly
 - Keeps the URL in sync with display
- Install with npm
 - 3 versions: core, web and native
 - We'll be using the dom version since we're building web apps

```
npm install react-router-dom
```

What does React Router Do?



- Easy way to add multiple levels of navigation
- Keeps url in sync with content
- Handles cross-browser inconsistencies
- Gives us Dynamic Routing
 - Routing happens as the application is rendering, not beforehand
 - This makes it so that almost anything inside the library is a Component

Getting Started with React-Router



- The main components in react-router are the following:
 - **Router components:** should be at the core of your application, keep the UI in sync with the url
 - **Route matching components:** supply conditional rendering of components based on the current location's pathname
 - **Navigation components:** Give you a way to navigate from one location to another

Router components



- 2 options: ~~BrowserRouter~~ **BrowserRouter** or **HashRouter**
 - Generally speaking, use **BrowserRouter** if you have a server that responds to requests, use **HashRouter** if you have a static file server
- Where do I add this component?
 - Should be at the core of your application, often in index.js
 - Wrap your main component inside a **BrowserRouter** component

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';
import {BrowserRouter } from 'react-router-dom';

ReactDOM.render(<BrowserRouter><App /></BrowserRouter>,
document.getElementById('root'));
```

Route Matching Components



- The **Route** component matches a certain path to determine if it should render its content

```
const RoutingExample = () => (  
  <div>  
    <Route path="/" component={Home}/>  
    <Route path="/about" component={About}/>  
    <Route path="/topics" component={Topics}/>  
  </div>  
)
```

- Content can be defined in 3 props:
 - **component**: pass a component to be rendered
 - **render**: pass a function that returns the content (convenient for inline rendering)
 - **children**: like render, but also gets called when there's no match (useful for animations or just showing if there's a match)

Route Matching Components



- **path**: Pattern used to match a given url
 - Path-to-regexp package

- Special symbols
 - :paramName
 - ? → optional param
 - * → like in regexp
 - + → like in regexp

```
//matches /hello/michael and /hello/ryan  
<Route path="/hello/:name">  
//matches /hello, /hello/michael, and /hello/ryan  
<Route path="/hello/:name?">  
//matches /files/hello.jpg and /files/hello.html  
<Route path="/files/.*\\.html">
```

Route Matching Components



- Additional props for the Route Component:
 - **exact**: when true, will only match when the path matches exactly
 - **strict**: when true, trailing slashes will be taken into account for matching
 - **sensitive**: when true, path matching becomes case-sensitive
- Watch out with the "/" path!

```
//in this case, "/about" will render both
the Home and About component
<Route path="/" component={Home} />
<Route path="/about" component={About} />
<Route path="/inbox" component={Inbox} />
```

```
//after adding "exact", everything works as
expected
<Route exact path="/" component={Home} />
<Route path="/about" component={About} />
<Route path="/inbox" component={Inbox} />
```

Route Matching Components



- By default, Routes are independent of each other
 - If multiple routes match, multiple components will be shown
- If you want to only show the first match, you can use a **Switch** component
- Example:

```
<Switch>
  <Route exact path="/" component={Home} />
  <Route path="/about" component={About} />
  <Route path="/contact" component={Contact} />
  { /* when none of the above match, <NoMatch> will be rendered */ }
  <Route component={NoMatch} />
</Switch>
```

Navigation Components



- **Link** component: provides a link to a route

```
<Link to="/">Home</Link>  
// <a href="/">Home</a>
```

- **NavLink** component: a special kind of Link that can style itself when its "to" prop matches the current path

```
<NavLink to="/about" activeClassName="selected">About</NavLink>  
// <a href="/react" className="selected">React</a>
```

- **Redirect** component: when you want to force navigation, you can render a Redirect

```
<Redirect to="/login"/>
```

Routing With React-Router: Example



```
JSX  
class App extends React.Component {  
  render() {  
    return (  
      <div>  
        <h1>App</h1>{/* change the <a>s to <Link>s */}  
        <ul>  
          <li><Link to="/">Home</Link></li>  
          <li><Link to="/about">About</Link></li>  
          <li><Link to="/inbox">Inbox</Link></li>  
        </ul>  
      </div>  
      <Route exact path="/" component={Home} />  
      <Route path="/about" component={About} />  
      <Route path="/inbox" component={Inbox} />  
    );  
  }  
}
```


Routing With React-Router: Example



JSX

```
ReactDOM.render(  
  <Router>  
    <App/>  
  </Router>  
, document.body);  
})();
```

// Don't forget these lines!

```
import { BrowserRouter as Router, Route, Link } from 'react-router-dom';
```

7.3 NESTED ROUTES

Nested Routes



- Nested routes allow us to have multiple levels of navigation
- Imagine the following:
 - **App** component contains routes to **Home** and **ProductList** components
 - **ProductList** component contains routes to multiple **ProductDetails** components
- In the App component the path for the products route is **/products**
- How do we build the routes to the ProductDetails components in the Products component?
- We could write out the entire route e.g.: **/products/:id**
 - But what if we need to show the Products component somewhere else as well?
 - Use **this.props.match!**

Nested Routes



- Every route component has access to a **match** object
- Contains information on how the Route matched the url.
 - **params**: Key/value pairs parsed from the URL corresponding to the dynamic segments of the path
 - **isExact**: true if the entire URL was matched (no trailing characters)
 - **path**: The path pattern used to match (useful for nested **Routes**)
 - **url**: The matched portion of the URL (useful for nested **Links**)
- Access to match differs depending on the type of match object
 - **component**: `this.props.match`
 - **render & children**: input param e.g.: `({match}) => ()`

Nested Routes



▪ Match Example:

```
//ProductList.js
const ProductList = ({ match }) => (

  <div>
    <h3>Products</h3>
    <Link to={` ${match.url}/1`} >
      Product 1
    </Link>
    <Link to={` ${match.url}/2`} >
      Product 2
    </Link>
    ...
    <Route path={` ${match.path}/:productId`}
      component={Product} />
  </div>
);
```

```
//Product.js
const Product = ({ match }) => {

  let productId = match.params.productId;
  //mock fetching product from datasource
  let product = {name: 'Product 1',
    description: 'lorem ipsum...',
    price: 19.99 };

  return (
    <div>
      <h4>{product.name}</h4>
      <p>{product.description}</p>
    </div>
  );
}
```

7.4 ADVANCED USAGE

Component Lifecycle



- When navigating the components will get updated
 - Looping through the lifecycle of multiple components
- Let's look at the following nested routing:
 - Not an actual code representation

```
<Route path="/" component={App}>  
  <Route exact path="/" component={Home}/>  
  <Route path="invoices/:invoiceId" component={Invoice}/>  
  <Route path="accounts/:accountId" component={Account}/>  
</Route>
```

- Important to understand these lifecycles
 - Especially when trying to update or fetch new data

Component Lifecycles



- The user enters at '/'

Component	Lifecycle Hooks called
App	componentDidMount
Home	componentDidMount
Invoice	N/A
Account	N/A

Component Lifecycles

U2U

- Navigate from / to /invoice/123

Component	Lifecycle Hooks called
App	componentWillReceiveProps, componentDidUpdate
Home	componentWillUnmount
Invoice	componentDidMount
Account	N/A

- App: re-rendered with new props
- Home: no longer rendered, so unmount
- Invoice: gets mounted

Component Lifecycles

U2U

- Navigate from /invoice/123 to /invoice/789

Component	Lifecycle Hooks called
App	componentWillReceiveProps, componentDidUpdate
Home	N/A
Invoice	componentWillReceiveProps, componentDidUpdate
Account	N/A

- Same components, just other properties

Component Lifecycles



- Navigate from /invoice/789 to /accounts/123

Component	Lifecycle Hooks called
App	componentWillReceiveProps, componentDidUpdate
Home	N/A
Invoice	componentWillUnmount
Account	componentDidMount

7.5 LAB – BUILDING A SPA WITH REACT AND REACT-ROUTER

In this lab, you will be building a Single Page application which is loosely based on the U2U website.

The application will contain a home and about component, as well as some components for browsing courses.

The end result should look something like this:



7.5.1 BEFORE YOU START

This lab starts from starter files found in the "single-page Starter" folder. Ask your trainer.

In the starter file you will find the following:

- **Home component:** a simple component that renders a dummy homepage.
- **About component:** just like the home component, this one shows a dummy about page.
- **Courses component:** This component shows a small selection of courses available at U2U Training.
- **data.js:** contains an array of courses, and an map object that assigns colors to each course category.

37. Run **npm install** to fetch the dependencies.

38. View the application by running **npm start**. (you can change the component that's currently rendered in App.js)

7.5.2 INSTALLING REACT-ROUTER

39. Since you're going to build a Single Page Application, you're going to need react-router. Install it using the following command:

COMMAND LINE

```
npm i --save react-router-dom
```

7.5.3 ADDING A NAVBAR

40. Create a new component called **Navigation**. In this component you will show a navbar containing links to the 3 existing components. Use the NavLink component so that you can see which link is active.

NAVIGATION.JSX

```
import React, { Component } from 'react';
import { NavLink } from 'react-router-dom';

class Navigation extends Component {
  render() {
    return (
      <nav>
        <div className="nav-wrapper black">
          <a href="#" className="brand-logo">U2U</a>
          <ul id="nav-mobile" className="right hide-on-med-and-down">
            <li><NavLink exact to="/">Home</NavLink></li>
            <li><NavLink to="/courses">Courses</NavLink></li>
            <li><NavLink to="/about">About</NavLink></li>
          </ul>
        </div>
      </nav>
    );
  }
}

export default Navigation;
```

7.5.4 ENABLING THE ROUTER

To get your routing to work, you'll first need to wrap your application in a BrowserRouter component.

41. In index.js, make sure you render a BrowserRouter around your app component.

INDEX.JS

```
import React from 'react';
import ReactDOM from 'react-dom';
import './index.css';
import App from './App';
import registerServiceWorker from './registerServiceWorker';
import { BrowserRouter } from 'react-router-dom';

ReactDOM.render(
  <BrowserRouter>
    <App />
  </BrowserRouter>, document.getElementById('root'));
registerServiceWorker();
```

42. Next, add the navbar to your app component and configure the routes beneath it.

APP.JS

```
import React, { Component } from 'react';
import Courses from './Courses';
import Navigation from './Navigation';
import About from './About';
import Home from './Home';
import { Route } from 'react-router-dom';

class App extends Component {
  render() {
    return (
      <React.Fragment>
        <Navigation />
        <div className="container">
          <Route exact path="/" component={Home} />
          <Route path="/courses" component={Courses} />
          <Route path="/about" component={About} />
        </div>
      </React.Fragment>
    );
  }
}

export default App;
```

This should give you a working application with basic navigation.

7.5.5 NAVIGATING THE COURSES

If you thought we were going to stop here, I've got news for you.

You want to be able to navigate the courses as follows:

- **/courses**: show a list of all courses.
- **/courses/:code**: show the detail view of a specific course.
- **/category/:category**: show all courses within a specific category.

43. First, you'll create a course detail component that will show the details of a given component. The course you'll show will be determined by the code parameter in the url.

COURSEDETAIL.JSX

```
import React, { Component } from 'react';
import { courses } from './data';

class CourseDetail extends Component {
  render() {
    const course = courses.find(c => c.code === this.props.match.params.code);
    return (
      <React.Fragment>
        <div className="card">
          <div className="card-content">
            <span className="card-title">{course.name}</span>
            <p><strong>category:</strong> {course.category}</p>
            <p><strong>duration:</strong> {course.days}</p>
            <p><strong>price:</strong> €{course.price}</p>
          </div>
        </div>
      </React.Fragment>
    );
  }
}

export default CourseDetail;
```

44. Now link to this component from the Courses component.

COURSES.JSX

```
import React, { Component } from 'react';
import { courses, colors } from './data';
import { Link } from 'react-router-dom';

class Courses extends Component {
  render() {
    const list = courses.map((c) => <li key={c.code} className="collection-item">
      <Link to={`/${this.props.match.url}/${c.code}`}>
        {c.name}
      </Link>
      <span className={`badge ${colors[c.category]} white-text`}>{c.category}</span>
    </li>);
    return (
      <React.Fragment>
        <h2>U2U Training</h2>
        <h3>Courses</h3>
        <ul className="collection">
          {list}
        </ul>
      </React.Fragment>
    );
  }
}

export default Courses;
```

45. Enable the route by adding it to App.js. Mind the order of the routes.

APP.JS

```

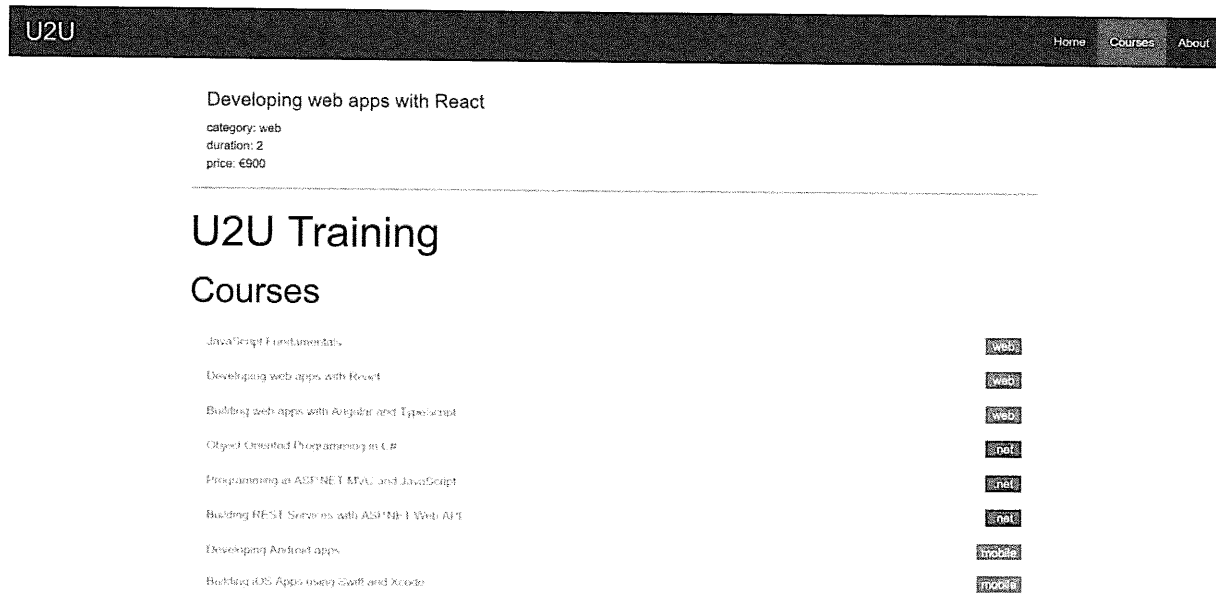
import { Route } from 'react-router-dom';
import CourseDetail from './CourseDetail';

class App extends Component {
  render() {
    return (
      <React.Fragment>
        <Navigation />
        <div className="container">
          <Route exact path="/" component={Home} />
          <Route path="/courses/:code" component={CourseDetail} />
          <Route path="/courses" component={Courses} />
          <Route path="/about" component={About} />
        </div>
      </React.Fragment>
    );
  }
}

export default App;

```

When you test this, you'll see something like this:



Can you guess what's going wrong? Both routes are matching, so both components are rendered.

46. Add a Switch around your routes so that only the first match gets rendered.

APP.JS

```

import { Route, Switch } from 'react-router-dom';
import CourseDetail from './CourseDetail';

class App extends Component {
  render() {
    return (
      <React.Fragment>
        <Navigation />
        <div className="container">
          <Switch>
            <Route exact path="/" component={Home} />
            <Route path="/courses/:code" component={CourseDetail} />
            <Route path="/courses" component={Courses} />
            <Route path="/about" component={About} />
          </Switch>
        </div>
      </React.Fragment>
    );
  }
}

export default App;

```

7.5.6 SHOWING COURSES BY CATEGORY

Almost there. To finish this lab, you're going to add a Category component which will show all courses for a given category. You can use the badges in the course overview to navigate to this component.

47. Create the Category component. (You can reuse a lot of code from the Courses component)

CATEGORY.JSX

```

import React, { Component } from 'react';
import { courses, colors } from './data';
import { Link } from 'react-router-dom';

class Category extends Component {
  render() {
    const filteredCourses = courses.filter(c => c.category ===
this.props.match.params.category);
    const list = filteredCourses.map((c) => <li key={c.code}
className="collection-item">
      <Link to={`/courses/${c.code}`}>
        {c.name}
      </Link>
    </li>);
    return (
      <React.Fragment>
        <h2>U2U Training</h2>
        <h3>Courses in <span
className={` ${colors[this.props.match.params.category]}-
text`}>{this.props.match.params.category}</span></h3>
        <ul className="collection">
          {list}
        </ul>
      </React.Fragment>
    );
  }
}

export default Category;

```

48. Link to the Category component from the Courses component.

COURSES.JSX

```

...
render() {
  const list = courses.map((c) => <li key={c.code} className="collection-
item">
    <Link to={`/${this.props.match.url}/${c.code}`}>
      {c.name}
    </Link>
    <Link to={`/category/${c.category}`}>
      <span className={ badge ${colors[c.category]} white-
text`}>{c.category}</span>
    </Link>
  </li>);
  return (
    <React.Fragment>
      <h2>U2U Training</h2>
      <h3>Courses</h3>
      <ul className="collection">
        {list}
      </ul>
    </React.Fragment>
  );
}
...

```

49. Add the route to App.js

APP.JS

```

import Category from './Category';
...
render() {
  return (
    <React.Fragment>
      <Navigation />
      <div className="container">
        <Switch>
          <Route exact path="/" component={Home} />
          <Route path="/category/:category" component={Category} />
          <Route path="/courses/:code" component={CourseDetail} />
          <Route path="/courses" component={Courses} />
          <Route path="/about" component={About} />
        </Switch>
      </div>
    </React.Fragment>
  );
}
...

```

That's it, you now have a working application. Sort of... Take a look at the extra challenge to see what you're missing.

7.5.7 (OPTIONAL) EXTRA CHALLENGE: ALLOWING BACK-NAVIGATION

At the moment, you can only navigate downwards in your application. Add links from the CourseDetail component back to the overview, and to the category view for the current course's category.

Also add a link back to the overview from the category view.

8 REACT INTERNALS

Agenda



- Getting references for components
- Ref Callback
- Prop-Types

8.1 REFS

Refs



- Reference towards DOM node or Component
- Used for
 - Finding and changing markup on a DOM element
 - Using a React Component in a non-React application
- You can
 - Call functions on the component
 - Read/Write DOM nodes

Instance of Component



- ReactDOM.render() returns the instance
 - Don't confuse it with the render() function in a class!

```
var myComponent = ReactDOM.render(  
  <MyTextBox />, document.getElementById('example')  
);  
myComponent.executeMethodOnComponent();
```

- Should only be used for top level component
 - Use props or state to communicate with child components
- Try to always follow the Data Flow hierarchy!

8.2 REF CALLBACK

Using Ref Callback



- Reference towards an element
 - Inside the Component Hierarchy
- Examples:
 - Getting focus dynamically when component mounts
 - Getting data from a field not related to the current event

Using Ref Callback



- Attaching the ref to a DOM component (`<div />`)
 - Returns the DOM node
- Attaching ref to component (`<TextInput />`)
 - Returns the instance
 - Allows calling methods on that object

How To?



JSX

// Here is an example to get a reference for a component

```
class SpecialBox extends React.Component{
  render() {
    return (
      // This function gets a reference towards the current "TextInput" element
      <TextInput ref={function(input) { if (input != null) { input.focus(); }}} />
    );
  },
}
// This technique could be used to reference the TextInput from anywhere
```

Example With an HTML Element



JSX

```
class MyComponent extends React.Component{
  handleClick = () => {
    // Explicitly focus the text input using the raw DOM API.
    this.myTextInput.focus();
  }
  render() {
    return (
      <div>
        <input type="text" ref={(ref) => this.myTextInput = ref} />
        <input type="button" value="Focus text" onClick={this.handleClick} />
      </div>
    );
  }
}

ReactDOM.render( <MyComponent />, document.getElementById('example'));
```

React.createRef()



- Introduced in React 16.3
- Preferred over ref callbacks (when available)

```
class CustomTextInput extends React.Component {
  constructor(props) {
    super(props);
    this.textInput = React.createRef();
    this.focusTextInput = this.focusTextInput.bind(this);
  }
  focusTextInput() {
    this.textInput.current.focus();
  }
  render() {
    return (
      <div>
        <input type="text" ref={this.textInput} />
        <input type="button" value="Focus the text input" onClick={this.focusTextInput} />
      </div>
    );
  }
}
```

8.3 PROPERTY TYPES

Property Types



- Ensure components are used correctly
 - Specifying propTypes is helpful
- PropTypes exports validators
 - Shows warnings in the JS console
 - Only checked in development mode!

```
import PropTypes from 'prop-types';

class ProfileLink extends React.Component{
  /* ... */
}

ProfileLink.propTypes = {
  /* ... */
}
```

Types of Props



```
JSX

class Todo extends Component {
  ...
  render() {
    return (
      <p ref="para">
        <input type="checkbox" ref="check" id={this.props.item.text} onChange={this.handleChange} checked={this.props.item.isDone} />
        <label className={this.props.item.isDone ? "done" : "open"} htmlFor={this.props.item.text}>
        <span>{this.props.item.text}</span></label>
      </p>
    );
  }
}

Todo.propTypes = {
  item: PropTypes.shape({
    text: PropTypes.string.isRequired,
    isDone: PropTypes.bool,
  }),
  action: PropTypes.func
};

export default Todo;
```

Types of Props



- You can declare that a prop is a specific JS primitive. By default, these are all optional.

```
optionalArray: React.PropTypes.array,  
optionalBool:  React.PropTypes.bool,  
optionalFunc:  React.PropTypes.func,  
optionalNumber: React.PropTypes.number,  
optionalObject: React.PropTypes.object,  
optionalString: React.PropTypes.string,
```

- Anything that can be rendered: numbers, strings, elements or an array (or fragment) containing these types.

```
optionalNode: React.PropTypes.node,
```

Types of Props



- You can also declare that a prop is an instance of a class. This uses JS's instanceof operator.

```
optionalMessage: React.PropTypes.instanceOf(Message),
```

- You can ensure that your prop is limited to specific values by treating it as an enum.

```
optionalEnum: React.PropTypes.oneOf(['News', 'Photos']),
```

- An object that could be one of many types

```
optionalUnion: React.PropTypes.oneOfType([  
  React.PropTypes.string,  
  React.PropTypes.number,  
  React.PropTypes.instanceOf(Message)  
]),
```

Types of Props



- An array of a certain type

```
optionalArrayOf: React.PropTypes.arrayOf(React.PropTypes.number),
```

- An object with property values of a certain type

```
optionalObjectOf: React.PropTypes.objectOf(React.PropTypes.number),
```

- An object taking on a particular shape

```
optionalObjectWithShape: React.PropTypes.shape({  
  color: React.PropTypes.string,  
  fontSize: React.PropTypes.number  
})),
```

- You can chain any of the above with `isRequired` to make sure a warning is shown if the prop isn't provided.

```
requiredFunc: React.PropTypes.func.isRequired,
```

Types of Props



- A value of any data type

```
requiredAny: React.PropTypes.any.isRequired,
```

- You can also specify a custom validator. It should **return** an **Error** object if the validation fails.

```
customProp: function (props, propName,  
  componentName) {  
  if (!/matchme/.test(props[propName])) {  
    return new Error('Validation failed!');  
  }  
}
```

Single Child



JSX

```
class MyComponent extends React.Component{
  render() {
    return (
      <div>
        // This must be exactly one element or it will warn.
        {this.props.children}
      </div>
    );
  }
}
MyComponent.propTypes = {
  children: React.PropTypes.element.isRequired
}
```

8.4 DEFAULT PROP VALUES

Default Prop Values



- Define default values for props in a very declarative way

```
class Greeting extends React.Component {  
  render() {  
    return (  
      <h1>Hello, {this.props.name}</h1>  
    );  
  }  
}  
  
// Specifies the default values for props:  
Greeting.defaultProps = {  
  name: 'Stranger'  
};
```

- `defaultProps` is cached
 - ensures `this.props.name` has a value

9 REDUX

Agenda



- What is Redux?
- Actors
- Tools

9.1 WHAT IS REDUX?

What is Redux?

Used between unrelated components

U2U

- A State Management pattern
 - Based on Flux
- Goals are
 - minimal API
 - predictable behavior
 - hot reloading *→ only ui reload and not state*
 - time travel
 - record and replay

Based on Three Principles

U2U

- Single source of truth
 - The state of the application is stored in object tree within single store
- State is read-only
 - The only way to change state is to emit an action
- Changes are made with pure function
 - To specify how the state tree is transformed by actions, write pure reducers

Installing Redux



- Using npm of course...

```
npm install --save redux
```

- Complementary packages for react

```
npm install --save react-redux
npm install --save-dev redux-devtools
```

Why Redux?

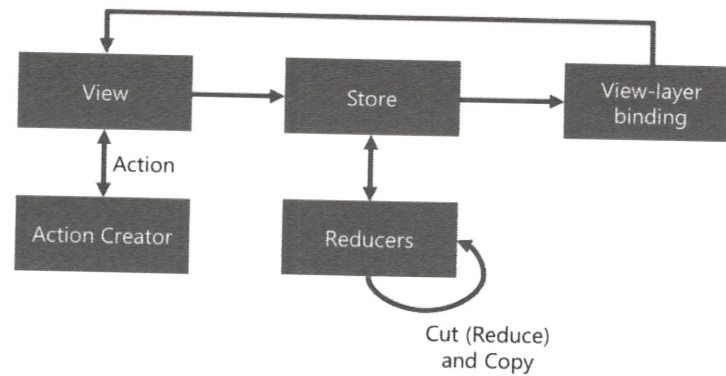


- A SPA contains a lot of information
 - It manages a lot more state than traditional pages
 - UI is also more complex
 - Spinners, sidebars, pagination, ...
 - This is also state
 - New complex features
 - Undo/redo, optimistic updates, ...
- If anyone can change state at any time → Chaos
- **redux specifies constraints to make state mutation predictable**

9.2 ACTORS

Visualization

u2u



Action Creator

u2u

- Functions that create actions
 - Actions are just payloads of data and the type of the action
- The functions just return an action object:

```

function addTodo(text) {
  return {
    type: ADD_TODO,
    text
  }
}

```

→ the action you do
→ the payload

- NOTE: they do not dispatch the actions!



Reducer



- Specify how the application's state changes on a given action
- It is a pure function which
 - takes the previous state and an action
 - Returns the next state
- Note:
 - Do not mutate the state, a copy is created on which the update happens
 - In a case where the action is unknown, return the previous state

Example Reducer



```
JS
import { VisibilityFilters } from './actions';

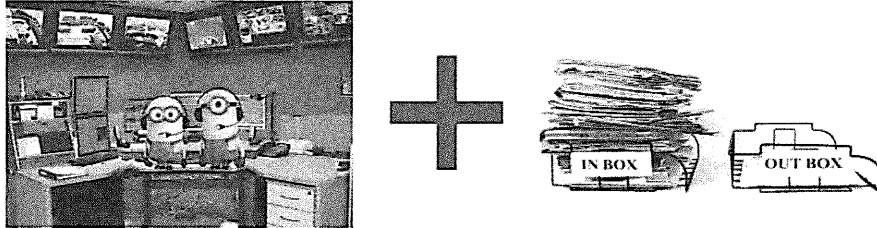
const initialState = {
  visibilityFilter: VisibilityFilters.SHOW_ALL,
  todos: []
};

function todoApp(state = initialState, action) {
  switch (action.type) {
    case SET_VISIBILITY_FILTER:
      return Object.assign({}, state, { visibilityFilter: action.filter });
    default:
      return state;
  }
}
```

Store

U2U

- There is only one store object which
 - Holds application state
 - Allows access to the state
 - Allows state to be updated (through a dispatch)
 - Registers and unregisters listeners
- To create a store, a reference to the root reducer is necessary



Example Store

U2U

```
JS
import { createStore } from 'redux'
import todoApp from './reducers'
let store = createStore(todoApp)

// In a view
store.dispatch(addTodo('learn about actions'));
store.dispatch(addTodo('learn about reducers'));
store.dispatch(addTodo('learn about store'));
store.dispatch(setVisibilityFilter(VisibilityFilters.SHOW_COMPLETED));
```

View-Layer Binding



- Connects the store with the views
 - In react this is react-redux
- IT department for the views
 - Makes sure all components can connect to the store
- Introduces three concepts
 - Provider component providing a way to hook to the store
 - Connect function: uses a selector to set up the connection for the store
 - Selector function: what parts of the state does the component need



View



- Display data from the store
- Components are the views in React
 - Smart and dumb components
- Smart component:
 - In charge of the actions
 - No css styles
 - Arrange dumb components on the page instead of actual HTML
- Dumb components
 - Show DOM
 - Use logic coming from the smart component

Microsoft and Apple Training



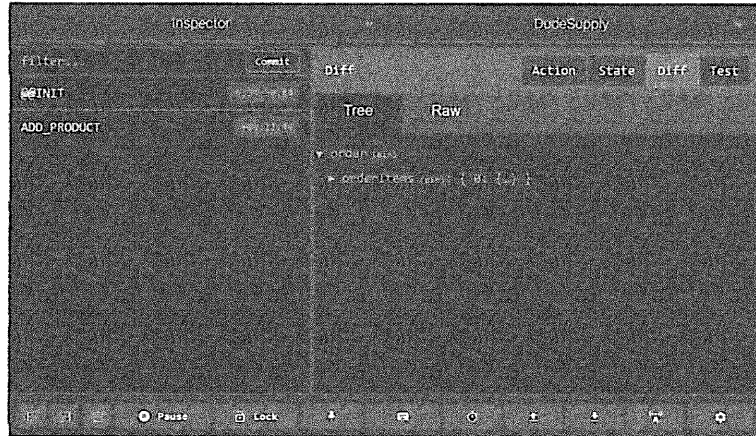
Demo

9.3 TOOLS

Redux DevTools

u2u

- Chrome Extension



Redux DevTools

u2u

- Features
 - View application state
 - View actions
 - Time travel
 - Dispatch your own action
 - Hot reloading
- Requirements
 - You need to add a little bit of code to your project to inform the tool

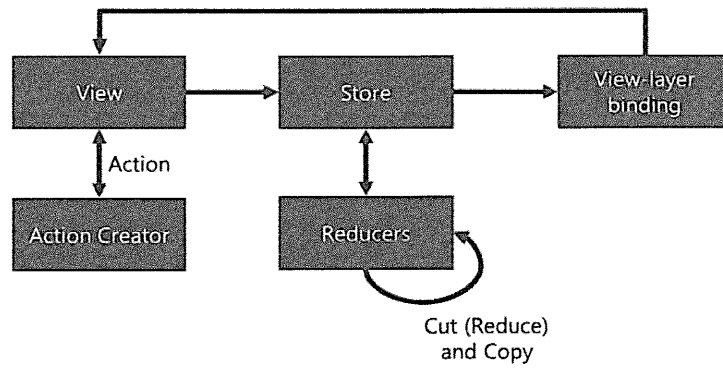
```
const store = createStore(reducer,
window.__REDUX_DEVTOOLS_EXTENSION__ && window.__REDUX_DEVTOOLS_EXTENSION__());
```


9.4 SUMMARY

Summary

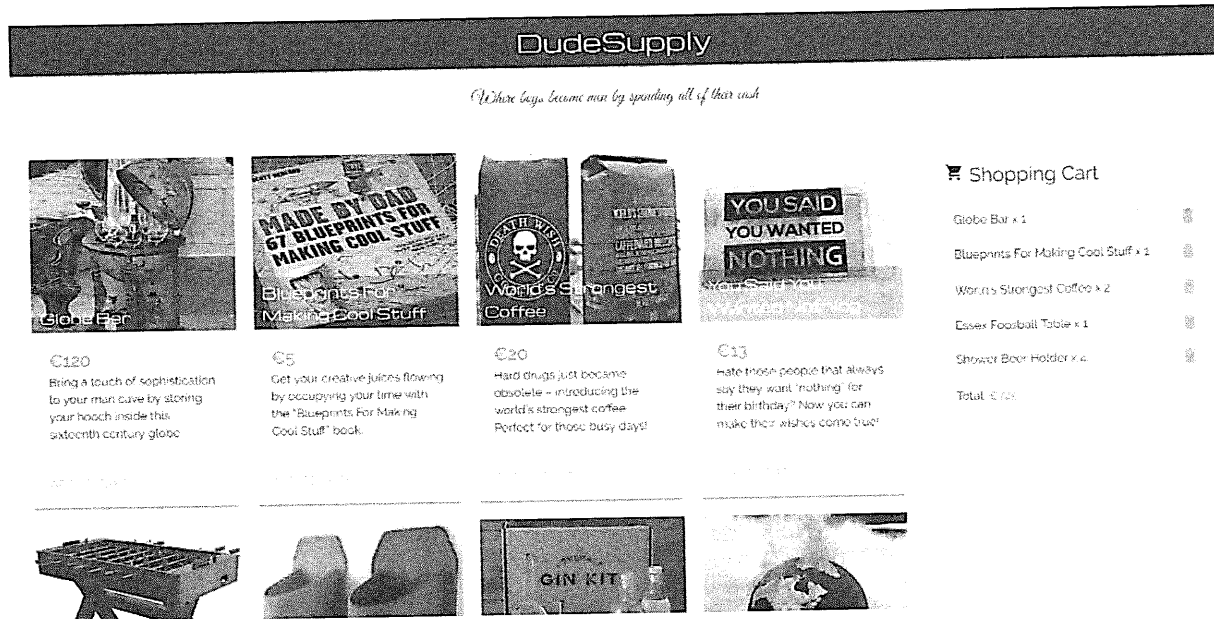
u2u

- What is Redux?
- Actors
- Tools



9.5 LAB – CREATING A WEBSHOP WITH REACT AND REDUX

In this lab you will create a webshop called DudeSupply. The end result will look something like this:



9.5.1 BEFORE YOU START

This lab starts from starter files found in the "Redux Webshop Starter" folder. Ask your trainer.

In the starter files you will find a static version of the webshop, meaning the components are already there, but there isn't any behavior yet.

9.5.2 INSTALLING REDUX

50. To use Redux in React, you'll first install a couple of libraries:

COMMAND LINE

```
npm install --save redux react-redux
```

9.5.3 CREATING THE REDUCERS

Usually, you'll start by building your reducers and combining them into a root reducer. Start off by taking a step back and thinking about what you'll need

9.5.3.1 DEFINING THE ACTION TYPES

You know your reducers are going to be dealing with 2 things: a list of products, and an order which contains some order items. The actions you're going to need are the following:

- Load Products
- Add Product to order
- Remove Product from order

51. Create a matching set of action constants in a new file under `src/constants/ActionTypes.js`

ACTIONTYPES.JS

```
export const DELETE_PRODUCT = 'DELETE_PRODUCT';
export const ADD_PRODUCT = 'ADD_PRODUCT';
export const LOAD_PRODUCTS = 'LOAD_PRODUCTS';
```

9.5.3.2 CREATING ACTIONS FOR THE ACTION TYPES

Now you'll define the shape of your action functions. They will contain the action you want to perform, and the payload you'll pass to the reducer.

52. Create a new file under **src/actions/index.js** and add the action functions there

INDEX.JS

```
import * as types from '../constants/ActionTypes';

export const addProduct = product => ({ type: types.ADD_PRODUCT, product });
export const deleteProduct = (title) => ({ type: types.DELETE_PRODUCT, title });
export const loadProducts = () => { type: types.LOAD_PRODUCTS };
```

9.5.4 CREATING THE REDUCERS

Since you're working with 2 concepts, Products and orders, you'll create separate reducers for them, and combine them into a root reducer.

53. Start by creating the products reducer under **src/reducers/products.js**. Since the products are static in this application, you'll always return the default state.

PRODUCTS.JS

```
import { LOAD_PRODUCTS } from '../constants/ActionTypes';
import { data } from '../data.js';

const initialState = data.products;

export default function products(state = initialState, action) {
  switch (action.type) {
    case LOAD_PRODUCTS: // will only return a constant array of products
    default:
      return state;
  }
}
```

54. Now create the order reducer under **src/reducers/order.js**

ORDER.JS

```
import { DELETE_PRODUCT, ADD_PRODUCT } from '../constants/ActionTypes';

const initialState = { orderItems: [] };

export default function order(state = initialState, action) {
  switch (action.type) {
    case DELETE_PRODUCT:
      let newItem, newOrderItems;

      let item = state.orderItems.find(e1 => e1.product.name ===
action.title);

      if (!item) {
        return state; //product not found, return state as-is
      }

      newItem = {
        ...item,
        amount: item.amount - 1 //decrement amount
      };

      if (newItem.amount > 0) {
        newOrderItems = state.orderItems.slice(); //shallow copy of
array
        let itemIndex = state.orderItems.indexOf(item);
        newOrderItems[itemIndex] = newItem; //replace item with
newItem
      } else {
        newOrderItems = state.orderItems.filter(e1 => e1 !==
item); //amount === 0, remove from list
      }

      return {
        orderItems: newOrderItems
      };

    case ADD_PRODUCT:
      let newItem = { product: action.product, amount: 1 };
      let newOrderItems;

      let item = state.orderItems.find(e1 => e1.product.name ===
action.product.name);
      if (item) {
        newItem.amount = item.amount + 1; //increment amount
        newOrderItems = state.orderItems.slice(); //shallow copy of array
        let itemIndex = state.orderItems.indexOf(item);
        newOrderItems[itemIndex] = newItem; //replace item with newItem
      } else {
        newOrderItems = state.orderItems.concat(newItem);
      }
      return {
        orderItems: newOrderItems
      };

    default:
      return state;
  }
}
```

55. Finally, combine both reducers into a root reducer under **src/reducers/index.js**. You'll use the `combineReducers` helper function from `redux` for this.

INDEX.JS

```
import { combineReducers } from 'redux';
import order from './order.js';
import products from './products.js';

const rootReducer = combineReducers({
  order,
  products
});

export default rootReducer;
```

9.5.5 CREATING THE STORE

Now that you've got your reducers in place, it's time to create a store. The store will contain the state for your entire application, and manage updating it.

56. Under `src/index.js`, call the `createStore` function, passing in the reducer. You'll also pass an extra argument to hook up the Redux DevTools browser extension. Then you'll wrap your App in a provider, giving React access to the store.

INDEX.JS

```
import React from 'react';
import { render } from 'react-dom';
import { createStore } from 'redux'; // Creates the Central Redux Store
import { Provider } from 'react-redux'; // Makes sure the Store is connected
to React
import App from './containers/App.js';
import reducer from './reducers'; // Gets the root reducer to be used by the
store

//hook up the redux devtools browser extension
const store = createStore(reducer, window.__REDUX_DEVTOOLS_EXTENSION__ &&
window.__REDUX_DEVTOOLS_EXTENSION__());

render(
  <Provider store={store}>
    <App />
  </Provider>,
  document.getElementById('root')
);
```

9.5.6 HOOKING UP THE STORE

Now that React knows about the store, you can let your application know about it as well.

57. Implement functions called `mapStateToProps` and `mapDispatchToProps` which will return the redux state and dispatchers to your actions respectively. `MapStateToProps` will allow you to access the redux state in your components' props, `mapDispatchToProps` will give you access to the actions.
58. Attach everything to the app using the `connect` method.

APP.JS

```
import { bindActionCreators } from 'redux'
import { connect } from 'react-redux'
import * as ShopActions from '../actions'

import React, { Component } from 'react';
import './App.css';
import ProductList from '../components/ProductList';
import ShoppingCart from '../components/ShoppingCart';

class App extends Component {
  render() {
    let {order, products, actions} = this.props;
    return (
      <div className="row">
        <ProductList products={products} actions={actions}/>
        <ShoppingCart items={order.orderItems} actions={actions} />
      </div>
    );
  }
}

const mapStateToProps = state => ({ // Creates the props from the State object
  in the store
  order: state.order,
  products: state.products
})

const mapDispatchToProps = dispatch => ({ // Wraps the Action Creators with
  the dispatch function for the views to use
  actions: bindActionCreators(ShopActions, dispatch)
})

export default connect( // Makes sure to attach everything from the App to the
  previous two methods
  mapStateToProps,
  mapDispatchToProps
)(App)
```

9.5.7 TYING EVERYTHING TOGETHER

With everything in place, you can hook up your components to start using the store.

PRODUCTLIST.JSX

```
import React, { Component } from 'react';
import ProductDetail from './ProductDetail';

class ProductList extends Component {
  render() {
    let actions = this.props.actions;
    const products = this.props.products.map((p) => (<ProductDetail
key={p.name} product={p} addProduct={actions.addProduct}/>));
    return (
      <div className="col s12 m9">
        {products}
      </div>
    );
  }
}

export default ProductList;
```

PRODUCTDETAIL.JSX

```
import React, { Component } from 'react';
import './ProductDetail.css';

class ProductDetail extends Component {
  render() {
    const product = this.props.product;
    return (
      <div className="col s12 m3">
        <div className="card">
          <div className="card-image">
            <img src={'/img/' + product.productImage} />
            <span className="card-title">{product.name}</span>
          </div>
          <div className="card-content">
            <span className="productPrice light-green-
text">&euro;{product.price}&nbsp;</span>
            <p>{product.description}</p>
          </div>
          <div className="card-action">
            <a onClick={() => this.props.addProduct(this.props.product)}>Add
To Cart</a>
          </div>
        </div>
      </div>
    );
  }
}

export default ProductDetail;
```

SHOPPINGCART.JSX

```
import React, { Component } from 'react';
import OrderLine from './OrderLine';
import './ShoppingCart.css';

class ShoppingCart extends Component {
  render() {
    var totalPrice = 0;
    this.props.items.forEach((element, index) => {
      totalPrice += element.product.price * element.amount;
    });
    const lines = this.props.items.map((i) => (<OrderLine key={i.product.name}
    item={i} deleteProduct={this.props.actions.deleteProduct}/>))
    return (
      <div className="col s12 m3">
        <ul className="collection with-header">
          <li className="collection-header"><h5><i className="material-
icons">shopping_cart</i> Shopping Cart</h5></li>
            {lines}
          <li className="collection-item"><h6>Total: <span
className="cartTotal">&euro;{totalPrice}</span></h6></li>
        </ul>
      </div>
    );
  }
}

export default ShoppingCart;
```

ORDERLINE.JSX

```
import React, { Component } from 'react';

class OrderLine extends Component {
  render() {
    let item = this.props.item;
    return (
      <li className="collection-item">{item.product.name} &times;
      {item.amount}<a onClick={() => this.props.deleteProduct(item.product.name)}
      className="secondary-content orange-text text-accent-2"><i
className="material-icons">delete</i></a></li>
    );
  }
}

export default OrderLine;
```

59. Check out the result by running the application!

10 JEST

Agenda



- What is Jest
- Creating tests
- Asynchronous Methods
- Mocking
- Jest and React

10.1 WHAT IS JEST

What is Jest



- Testing framework based on Jasmine
 - using familiar `expect(value).toBe(other)` assertions
- Jest provides multiple layers on top of Jasmine:
 - Automatically finds tests to execute
 - Automatically mocks dependencies
 - Allows asynchronous code to be tested synchronously
 - Runs tests with fake DOM implementation (jsdom)
 - Runs tests in parallel processes

Installing Jest



- Download the node package

```
npm install --save-dev jest
npm install --save-dev react-test-renderer
```
- Tests next to the components (other options available)
 - Call them `<component-file>.test.js[x]`
- Add a package.json rule

```
"scripts": {
  "test": "jest"
}
```
- Run the tests

```
npm test
```

10.2 CREATING TESTS

Creating Tests



```
JS
// sum.js
export default function sum(value1, value2) {
  return value1 + value2;
}

// sum.test.js
import sum from './sum';
describe('sum', function() {
  test('adds 1 + 2 to equal 3', function() {
    expect(sum(1, 2)).toBe(3);
  });
});
```

Jest Tests



- Jest runs tests found
 - in `__tests__` directories
 - Files called `[name].spec.js[x]`
 - Files called `[name].test.js[x]`
- Jest can mock functions and modules
 - Manually or semi-automatically

Running the Tests



- Run "npm test"
 - Should show similar output like below

```
> jest

PASS  src\sum.test.js
  ✓ renders without crashing (32ms)

Test Suites: 1 passed, 1 total
Tests:      1 passed, 1 total
Snapshots:  0 total
Time:       0.886s, estimated 1s
Ran all test suites.
```

10.3 ASYNCHRONOUS METHODS

How Do You Test an Async Method?



```
JS
// fetchuser.js
import $ from 'jquery';

function parseUserJson(userJson) {
  return {
    loggedIn: true,
    fullName: userJson.firstName + ' ' + userJson.lastName
  };
}

export default function fetchUser() {
  return $.ajax({
    type: 'GET',
    url: 'http://example.com/currentUser',
  }).then(data => parseUserJson(data));
}
```

Test the Call...



```
JS
// __tests__/fetchuser-test.js
import fetchUser from './fetchuser';
import $ from 'jquery';
jest.mock('jquery');

describe('fetchuser', function () {
  test('calls into $.ajax with the correct params', function (done) {

    // Call into the function we want to test
    fetchUser().then(() => { done(); });

    // Now make sure that $.ajax was properly called
    expect($.ajax).toHaveBeenCalledWith({
      type: 'GET',
      url: 'http://example.com/currentUser',
      success: jasmine.any(Function)
    });
  });
});
```

..., Was It Called?



- `fetchUser()` should be called
 - And it should properly start a call to `$.ajax`
- Since `$.ajax` is another module, Jest will auto-mock it
 - `$.ajax` mock is checked to see if parameters are correct
- How do I know if the callback was called afterwards?

Does It Call Back?



```
JS
test('calls the callback when $.ajax requests are finished', function () {
  expect.assertions(1);

  fetchUser().then(data => {
    expect(data).toEqual({
      loggedIn: true,
      fullName: 'Bobby '); DROP TABLE Users; --'
    });
  });

  // Emulate the process by which `$.ajax` would execute its own callback
  $.ajax.mock.calls[0 /*first call*/][0 /*first argument*/].success({
    firstName: 'Bobby',
    lastName: '' ); DROP TABLE Users; --'
  });
});
```


What Happens?



- Jest has mocked \$.ajax
 - Computing the result from fetchUser will call \$.ajax, which can be inspected
- Semi-Auto-mocking from jQuery?
 - Jest requires the real module, inspects it and then builds mock
- Mock functions have .mock property
 - Stores all interactions (like mock.calls to mock return values)
- These were tests for asynchronous functions
 - But the code is entirely synchronous!!

10.4 MOCKING

Unit Testing



- Writing effective unit tests
 - Means isolating units of code and testing that unit
- In JS this is commonly a module
 - Jest makes isolating a module from dependencies very easy
- Jest can generate mocks for the dependencies
 - `jest.genMockFromModule()`

Manual Mocking



- Create a new directory called `__mocks__` on the same level of the dependency
 - In the case of `node_modules`, on the same level as that folder
- In the `__mocks__` folder create a file with the same name as that of the dep
- In the test use

```
const fs = jest.mock('fs');
```

10.5 JEST AND REACT

Test Utilities



- Testing React Components
 - Use Snapshot testing
 - Use Enzyme
- Snapshot testing allows to take snapshots of your components
- Enzyme allows to create coded UI tests

Simulate



- Possibly the single most useful function from Enzyme for UI testing

```
simulate('eventName', args?)
```

- Simulates event on DOM node
 - optional eventData

- Changing an input element

```
const checkbox = shallow(<CheckboxWithLabel labelOn="On" labelOff="Off" />);  
expect(checkbox.text()).toEqual('Off');  
checkbox.find('input').simulate('change');
```

Example Test Component



```
JS
// CheckboxWithLabel.js

import React from 'react';

export default class CheckboxWithLabel extends React.Component {

  constructor(props) {
    super(props);
    this.state = {isChecked: false};
  }

  onChange = () => {
    this.setState({isChecked: !this.state.isChecked});
  }

  render() {
    return (
      <label>
        <input type="checkbox" checked={this.state.isChecked} onChange={this.onChange} />
        {this.state.isChecked ? this.props.labelOn : this.props.labelOff}
      </label>);
  }
}
```

Test Example



```
JS
// __tests__/CheckboxWithLabel-test.js

import React from 'react';
import {shallow} from 'enzyme';
import CheckboxWithLabel from '../CheckboxWithLabel';

test('CheckboxWithLabel changes the text after click', () => {
  // Render a checkbox with label in the document
  const checkbox = shallow(<CheckboxWithLabel labelOn="On" labelOff="Off" />);
  expect(checkbox.text()).toEqual('Off');
  checkbox.find('input').simulate('change');
  expect(checkbox.text()).toEqual('On');
});
```

Snapshot Testing



- Uses an extra module called react-test-renderer
- renderer is used to
 - Render the component
 - Create a snapshot
- The first time the test runs, it will create a snapshot file
 - Afterwards the snapshots are tested against this file
 - Include the snapshot file with your source control!

Snapshot Testing



```
JS
// Link.react.js
import React from 'react';

const STATUS = { HOVERED: 'hovered', NORMAL: 'normal' };

export default class Link extends React.Component {

  constructor(props) {
    super(props);
    this.state = { class: STATUS.NORMAL };
  }

  _onMouseEnter = () => { this.setState({class: STATUS.HOVERED}); }
  _onMouseLeave = () => { this.setState({class: STATUS.NORMAL}); }

  render() {
    return (<a className={this.state.class} href={this.props.page || '#'} onMouseEnter={this._onMouseEnter}
      onMouseLeave={this._onMouseLeave}>
      {this.props.children}
    </a>);
  }
}
```

Test Example



```
JS
// Link.react-test.js
import React from 'react';
import Link from '../Link.react';
import renderer from 'react-test-renderer';

test('Link changes the class when hovered', () => {
  const component = renderer.create(<Link page="http://www.facebook.com">Facebook</Link>);
  let tree = component.toJSON();
  expect(tree).toMatchSnapshot();

  // manually trigger the callback
  tree.props.onMouseEnter();
  // re-rendering
  tree = component.toJSON();
  expect(tree).toMatchSnapshot();

  // manually trigger the callback
  tree.props.onMouseLeave();
  // re-rendering
  tree = component.toJSON();
  expect(tree).toMatchSnapshot();
});
```

Snapshot file



```
JS
// __tests__/__snapshots__/Link.react-test.js.snap
exports[`Link changes the class when hovered 1`] = `
  <a className="normal" href="http://www.facebook.com" onMouseEnter={{Function}}
    onMouseLeave={{Function}}>
    Facebook
  </a>
`;

exports[`Link changes the class when hovered 2`] = `
  <a className="hovered" href="http://www.facebook.com" onMouseEnter={{Function}}
    onMouseLeave={{Function}}>
    Facebook
  </a>
`;

exports[`Link changes the class when hovered 3`] = `
  <a className="normal" href="http://www.facebook.com" onMouseEnter={{Function}}
    onMouseLeave={{Function}}>
    Facebook
  </a>
`;
```




On-schedule in Brussels

Register online at www.u2u.net/calendar

You can easily register for one of our open training sessions.
Just call or mail us or register online on our website.



On-site in EMEA

An increasing amount of companies prefer on-demand training. U2U offers a simple and budget friendly formula for groups. Any of our courses can be organized on-demand or on-site. You can even combine contents from different courses into a custom tailored course to maximize your business needs.

U2U TAKES PRIDE IN ITS FLEXIBILITY

What you want: customization

All U2U courses listed on our course calendar can be organized on-site and on-demand. The content of the courses can be fully customized to your particular needs, selecting the content which is important for your business. The content of the course will be drawn up beforehand in close cooperation with you.

Where you want: at U2U or on-site

On-demand training can be organized on-site at your premises throughout whole Europe, Middle East and Africa or in our classrooms in Brussels. Our courses can be delivered in Dutch, English and French. If your infrastructure is insufficient U2U can offer you our mobile PC class (portables and beamers) in your office or in a seminar center near you.

U2U comes to you

U2U is specialized in the organization of on-site courses throughout whole Europe, Middle East and Africa (EMEA).

In the last 5 years U2U trainers were active in:

Belgium, France, Netherlands, Luxembourg, Germany, United Kingdom, Jersey, Ireland, Spain, Portugal, Switzerland, Austria, Italy, Denmark, Sweden, Norway, Finland, Iceland, Poland, Russia, Slovenia, Estonia, Lithuania, Czechia, Croatia, Macedonia, Romania, Hungary, Bulgaria, Cyprus, Greece, China, Vietnam, Australia, Tanzania, Kenya, South-Africa, USA, Turkey, Egypt, Iran, Kuwait, Oman, Qatar, Saudi Arabia, United Arab Emirates

